Behavior in higher-order languages

Admissible observers, relative pushouts, and the encoding of extended theories

C.B. Wells, Michael Stay, L.G. Meredith

f1r3fly.io

Second draft

Abstract

We derive contextual transition systems for programming languages specified as *lambda theories* — equational logics with binding and rewriting — using Milner's framework of relative pushouts. Our central move is to make the class of admissible observers a *parameter* of the construction. Where Leifer and Milner restrict which contexts may *react*, we additionally restrict which contexts may *observe*, and compute relative pushouts inside that restricted class rather than in the ambient theory. The resulting notion of $\mathcal{D}$-relative bisimilarity is a congruence for $\mathcal{D}$, which is the statement one actually wants: for a reflective calculus, congruence fails for arbitrary contexts and holds for the admissible ones; for an encoding, it fails for arbitrary target contexts and holds for the images of source contexts. We show that three apparently separate phenomena — the failure of congruence under reflection in the ρ-calculus, the requirement that an encoding be probed only by encoded contexts, and the correctness of compiling away built-in data types — are one theorem instantiated three times. The last of these is our practical motivation: a user of a language-definition platform should be able to specify a pure theory and know that extending it with built-in data types leaves their reasoning intact.

Contents

1 Introduction

F1R3FLY is a concurrent computing platform based on the reflective higher-order π-calculus [Meredith and Radestock, 2005]. Its language-definition layer, MeTTaIL, lets a user present a domain-specific language as a signature with equations and rewrite rules, and compiles that presentation into the ρ-calculus for execution. For this to be more than a syntactic convenience, the user needs a guarantee: that reasoning carried out in the language they defined remains valid in the calculus their program is actually run in.

This paper is about the shape of that guarantee, and about a single obstacle that recurs whenever one tries to state it.

1.1 The obstacle

Behavioral equivalence is always relative to a class of observers. A labelled transition system fixes that class by fixing the labels, and since Milner [1982] the labels have been supplied by hand. Leifer and Milner [2000] showed that they need not be: labels can be *derived* as the minimal contexts enabling a rewrite, minimality being a universal property (a relative pushout), and bisimilarity for the derived system is then automatically a congruence.

The difficulty is that "minimal context" presumes we know which contexts are eligible. Leifer and Milner already need one restriction here — a subcategory $\mathcal{D}$ of *reactive* contexts, those in which a rewrite may occur — but they take the eligible *observers* to be all of the ambient category. In every situation we care about, that is too many.

- **Reflection.** In the ρ-calculus, @ turns a process into a name and $*$ turns it back. Communication tests names for *equality*, not for bisimilarity. So if $p \approx q$ with $p \neq q$, the context $\mathtt{out}(@(-), z)$ separates them: bisimilarity is not a congruence for contexts with a hole under @. It is a congruence for the rest.

- **Encodings.** An encoding $[\![-]\!] : \mathrm{S} \to \mathrm{T}$ lands its image in a much larger theory. Arbitrary T-contexts can read the encoding's plumbing — name supplies, servers, dead branches — as observable structure, separating the images of source-equivalent programs. Only the images $[\![c]\!]$ of source contexts respect the intended abstraction.

- **Extension.** A theory with a built-in `Bool` can be compiled into one without, but the compiled conditional leaves an unreachable branch behind. It is unreachable only because no *encoded* context ever addresses it.

These are the same problem. In each case there is a class $\mathcal{D}$ of contexts for which the theory is well-behaved, a larger class for which it is not, and no way to state the good theorem without naming $\mathcal{D}$.

1.2 The move

We therefore take $\mathcal{D}$ as data, and compute relative pushouts inside it.

Given a lambda theory T and a subcategory $\mathcal{D}$ of admissible observers, a $\mathcal{D}$-relative IPO is a context in $\mathcal{D}$ that is minimal *among contexts in* $\mathcal{D}$ enabling a given rewrite — not minimal in T. These differ: the T-minimal enabler of a transition out of $[\![p]\!]$ is typically strictly smaller than any $[\![c]\!]$, because $[\![c]\!]$ carries plumbing that minimality in T strips away. Insisting on T-minimality thus forces observations outside the image; insisting on encoded contexts abandons minimality. Relative minimality keeps both.

We do not choose $\mathcal{D}$ arbitrarily. Section 4 shows that $\mathcal{D}$ is determined by a much simpler datum: an *observation relation*, a family $\approx_A$ of equivalences indexed by the types of the theory, built up from a choice at the ground types by the usual logical-relation clauses. An operation is admissible exactly when it preserves the relation. The ρ-calculus story then falls out of a single decision —

that $\approx_{Nm}$ is equality — and the encoding story falls out of another — that the observation relation on the target is the image of the one on the source.

1.3 Contributions

1. A presentation of programming-language specifications as **lambda theories**: categories with finite limits whose subobject fibration is cartesian closed, carrying a distinguished type Pr and rewriting as a *proposition* $(\rightsquigarrow) \rightarrowtail Pr \times Pr$. Types, terms, equations and rewrites live in one structure, and currying entailments gives rewriting under binders (§2).

2. An **open** form of the Leifer–Milner construction. Their reactive systems fix a distinguished object 0 as the domain of IPOs, restricting attention to ground terms; their proofs do not use it. Dropping it yields a transition system on open terms (§5).

3. **Observation relations and admissible observers** (§4): the class $\mathcal{D}$ derived from a logical relation on types, with admissibility as preservation.

4. **$\mathcal{D}$-relative pushouts**, the derived $\mathcal{D}$-relative transition system, and the $\mathcal{D}$-relative congruence theorem: $\sim_{\mathcal{D}}$ is a congruence for $\mathcal{D}$ (§5). We also give the construction in $\mathcal{D}/\approx$, where labels are compared coinductively rather than syntactically, which is what lets equations of a source theory be satisfied only up to bisimilarity in a target.

5. Three instantiations: reflection in the ρ-calculus (§6), encodings including π into ρ (§7), and extension by built-in data types (§8), where we prove that the equations of a finitary algebraic theory become weak bisimulations under a Scott-style encoding, worked in full for `Bool`.

1.4 Status of the results

This is a working draft, and we think it more useful to say plainly where each result stands than to present a uniform surface. PROVED means a proof is given here or is immediate from a cited result; SKETCH means the argument is given but a case analysis is incomplete; OBLIGATION means the statement is what we intend to prove and the proof is outstanding.

Result	**Where**	**Status**
Observation relation determines $\mathcal{D}$	Prop. 9	PROVED
@ is not admissible	Prop. 12	PROVED
$\mathcal{D}$-relative congruence (strong)	Thm. 25	PROVED
Existence of $\mathcal{D}$-relative IPOs	Prop. 27	SKETCH
Decomposition-closure of $[\![C_\pi]\!]$ in ρ	Ob. 28	OBLIGATION
$\mathcal{D}$-relative congruence (weak)	Thm. 26	OBLIGATION
Congruence for ρ, admissible contexts	Thm. 36	PROVED
Redex IPOs exist for ρ and π	Thm. 30	SKETCH
Replication elimination $\rho^! \to \rho$	Thm. 40	PROVED
$\pi \to \rho$ preserves behavior	Thm. 41	OBLIGATION
Extension equations become weak bisimulations	Thm. 54	SKETCH
The same, for `Bool`	Prop. 52	PROVED
Retraction $\mathrm{T} \otimes \mathrm{A} \to \mathrm{T}$ uniform in T	Ob. 55	OBLIGATION

1.5 What is deliberately absent

Earlier drafts of this material attempted the ambient calculus, the λ-calculus, name-passing λ, combinators, and a general theory of "weakened" theories with relative pushouts taken over an internal congruence. All are omitted.

The ambient calculus is omitted because it fails our hypotheses for a reason worth stating: the operation $n[-]$ does not commute with parallel composition, $n[p \mid q] \neq n[p] \mid q$, so the redex-IPO enumeration of §5 does not apply. Bonchi et al. [2009] show that the IPO semantics for ambients is in any case too fine and must be coarsened by barbs; the correct treatment is a separate paper. The λ-calculus is omitted because it has no parallel composition at all, and because Di Gianantonio et al. [2009] already treat it with second-order contexts. The internal-congruence construction is subsumed: §5.9 obtains the same effect by taking relative pushouts in $\mathcal{D}/\approx$, which needs no new framework.

2 Lambda theories

Algebraic structures, such as rings or automata, are defined by operations that satisfy equations. Such an operation has many inputs and one output, and thus can be described as a morphism $f : \prod A_i \to B$ from a product of types to a type. Hence *cartesian categories* have become the general setting for universal algebra [Adámek et al., 2010]. We denote such an operation as follows.

$$a_1 : A_1, ..., a_n : A_n \vdash f : B$$

Yet to model programming languages, a more expressive framework is needed, as a program may be an input to another program: a higher-order operation, whose inputs can be other operations. This is a morphism in a *closed* cartesian category, with both product types and *function types*, whose terms are formed by lambda abstraction and used in beta reduction [Lambek and Scott, 1988].

$$\frac{\Gamma, x : X \vdash c : A}{\Gamma \vdash \lambda x.c : [X \to A]} \qquad \frac{\Gamma \vdash t : X \quad \Gamma \vdash f : [X \to A]}{\Gamma \vdash f(t) : A}$$

To save space, we may write $[X \to Y]$ as $[X \backslash Y]$ and $X \times Y$ as X, Y.

For example in the π calculus [Milner, 1993], an input process hosts a function from names to processes, which can evaluate when in parallel with an output process on the same channel.

$$n : Nm, \lambda x.c : [Nm \backslash Pr] \vdash \mathtt{in}(n, \lambda x.c) : Pr \qquad \Gamma \vdash \mathtt{out}(n, a) \mid \mathtt{in}(n, \lambda x.c) \rightsquigarrow c(a)$$

The symbol $\rightsquigarrow$ denotes a *rewrite* from one program to another, as in a basic step of computation. This is a binary relation on programs, found in virtually every language specification, and presented in the style of Plotkin [2004]. Reasoning about programs involves a combination of types, operations, equations, and rewrites; yet most literature does not explicate their universe of discourse as one cohesive logical structure.

$$p : Pr, q : Pr \vdash (p \rightsquigarrow q) \text{ prop}$$

Categorical logic provides a canonical setting: the structure of *a logic over a type system* [Jacobs, 1999]. Rewriting is a *proposition* on pairs of programs, and dynamics is expressed by *entailments* thereof. As each predicate depends on a context of types, the category (poset) of predicates and entailments depends on the category of types and terms: together they form a **fibered category**.

$$\begin{array}{ccc} \mathrm{SubT} & \varphi \quad \vdash \quad \psi \\ \big\downarrow & \\ \mathrm{T} & S \xrightarrow{\quad f \quad} T \end{array}$$

The morphism above expresses that for any $s : S$, $\varphi[s]$ entails $\psi[f(s)]$, written $s : S \mid \varphi \vdash \psi[f]$.

Every category with finite limits T determines a fibered category of subobjects $\mathrm{SubT} \to \mathrm{T}$, in which an object over Γ is a subobject $\varphi \rightarrowtail \Gamma$, entailment is inclusion, and substitution is pullback. The internal language of this structure is equational logic; the inference rules are collected in Appendix A, together with the characterization theorem of Jacobs [1999] that licenses their use.

2.1 Meta-contexts and the rewrite proposition

We will define rewriting in terms of a distinguished type Pr and a proposition $(\rightsquigarrow) \rightarrowtail Pr \times Pr$. To define operations on rewrites, we introduce the following concepts.

Definition 1. *Consider the category of endofunctors* $\mathbb{C}\mathrm{at}(\mathrm{T}, \mathrm{T})$*. Define* $\mathrm{C}[\mathrm{T}]$ *to be the subcategory generated by* $- \times X$ *and* $[X \to -]$ *over all* $X : \mathrm{T}$*. We call such a composite a* ***meta-context****, and denote one by* $\langle - \rangle : \mathrm{T} \to \mathrm{T}$*, or* $\langle - \rangle_i$ *when there are multiple. These extend to fibered endofunctors* $- \wedge \top_X$ *and* $\top_X \Rightarrow -$*, the composites of which form* $\langle - \rangle : \mathrm{SubT} \to \mathrm{SubT}$ *for propositions.*

Since $A \cong 1 \times A$, the identity is a meta-context. Meta-contexts are equipped with the following isomorphisms.

$$\begin{aligned} A \times (B \times -) &\cong (A \times B) \times - \\ A \to (B \to -) &\cong (A \times B) \to - \\ A \to (B \times -) &\cong (A \to B) \times (A \to -) \end{aligned}$$

So in general, $\langle - \rangle = A \times [B \to -]$ for some $A, B : \mathrm{T}$. To define equations with binding operations, we need the monoidal strength st^A_B of each $[B \to -]$ with respect to products of T [Kock, 1972]:

$$\begin{array}{ccccc} & & [B \to A] \times [B \to -] & & \\ & \overset{\eta^A_B}{\nearrow} & & \overset{\cong}{\searrow} & \\ A \times [B \to -] & & \xrightarrow{\qquad st^A_B \qquad} & & [B \to (A \times -)] \end{array}$$

where $\eta^A_B : A \to [B \to A]$ is the currying of the projection $\pi_A : A \times B \to A$. This converts a term a with b not occurring free in a into a term $\lambda b.a$ constant at a.

Definition 2. *An* ***equational logic with binding and rewriting*** *is a category with finite limits such that* $\pi : \mathrm{SubT} \to \mathrm{T}$ *is cartesian closed, with a type* Pr *and a proposition* $(\rightsquigarrow) \rightarrowtail Pr \times Pr$ *with entailments: base rewrites* $\rightsquigarrow_B = \{\Gamma \mid \top \vdash p \rightsquigarrow q\}$*, and rewrite operations* $\rightsquigarrow_O$ *of the form below.*

$$\prod \langle \Gamma_i \rangle_i \mid \bigwedge \langle p_i \rightsquigarrow q_i \rangle_i \vdash f(\vec{p}) \rightsquigarrow f(\vec{q})$$

We shorten to the name ***lambda theory****, as in the classic example of the* λ*-calculus.*

Definition 3. *A* ***presentation*** *of a lambda theory* T *consists of:*

- *a set of* ***base types*** $\mathrm{T}_{\mathrm{type}} = \{A\}$*, including* Pr
- *a set of* ***operations*** $\mathrm{T}_{\mathrm{oper}} = \{\Gamma \vdash t : A\}$*, with* $A \in \mathrm{T}_{\mathrm{type}}$
- *a set of* ***propositions*** $\mathrm{T}_{\mathrm{prop}} = \{\varphi\}$*, including* $(p : Pr, q : Pr \vdash p \rightsquigarrow q)$
- *a set of* ***entailments*** $\mathrm{T}_{\mathrm{ent}} = \{\Gamma \mid \Theta \vdash \varphi\}$*, including* $\rightsquigarrow_B$ *and* $\rightsquigarrow_O$ *above.*

A theory is ***finitely presented*** *if all four sets are finite.*

The source l of a rewrite $l \rightsquigarrow r$ is called a **redex**. Rewrite operations are composed as follows, forming the set of **rewrite constructors** $\rightsquigarrow_C$.

$$\frac{f : (\rightsquigarrow_O)(\prod\langle Pr\rangle_i)}{f : (\rightsquigarrow_C)(\prod\langle Pr\rangle_i)} \qquad \frac{\{c_i : (\rightsquigarrow_C)(\Gamma_i)\} \quad f : (\rightsquigarrow_O)(\prod\langle Pr\rangle_i)}{f(\prod\langle c_i\rangle_i) : (\rightsquigarrow_C)(\prod\langle \Gamma_i\rangle_i)}$$

For the domains formed this way, we use $\langle\!\langle - \rangle\!\rangle = \prod\langle ... \prod\langle - \rangle_i ... \rangle_z$ for the functor $\mathrm{T}^{\Sigma ... \Sigma n_{i ... z}} \to \mathrm{T}$, and may overload the term meta-context to refer to these functors.

A **generic rewrite** is a substitution of base rewrites into a rewrite constructor, written $d(\vec{l})$. A **specific rewrite** is a substitution of ground terms into a generic rewrite, $d(l(\vec{x}))$.

Rewrite constructors are the *reactive contexts* of Leifer and Milner: the substitution of base rewrites into a rewrite constructor is a morphism in the slice category T/Pr. Their relative pushouts are the transitions that characterize behavior — once we have said which contexts are permitted to take part, which is the subject of §4.

Note 1. The combination of cartesian ($\times$) and closed ($\to$) structure on $\mathrm{SubT} \to \mathrm{T}$ does not appear in the standard hierarchy of logical structures [Johnstone, 2002].

cartesian	regular	coherent	heyting	topos
$(\top, \wedge, =)$	$(..., \exists)$	$(..., \bot, \vee)$	$(..., \Rightarrow, \forall)$	$(..., \to, \Omega)$

Yet it is useful and natural. Arguably the simplest thing one could mean by *higher-order* reasoning is that in the same way we form types of functions, we can form *propositions about proofs*. Just as one curries $f : A \times B \to C$ to $A \to [B \to C]$, the same is useful for entailments. The basic rewrite of the π calculus is the entailment

$$\begin{array}{ccc} \top_{Nm}, \top_{Nm}, \top_{[Nm\backslash Pr]} & \vdash & (\rightsquigarrow) \\ \downarrow & & \downarrow \\ Nm, Nm, [Nm\backslash Pr] & \xrightarrow{\circ(a,n)|\mathtt{i}(a,\lambda x.c),\ c[n]} & Pr, Pr \end{array}$$

which can be curried to a rewrite of *open* processes — continuations with a bound name.

$$\begin{array}{ccc} \top_{Nm}, \top_{[Nm\backslash Pr]} & \vdash & \top_{Nm} \Rightarrow (\rightsquigarrow) \\ \downarrow & & \downarrow \\ Nm, [Nm\backslash Pr] & \xrightarrow[\lambda a.(...)]{} & [Nm\backslash Pr, Pr] \end{array}$$

This is how we reason about rewrites under a lambda and within binding constructors. It is also why the transition system of §5 can be defined on open terms without the distinguished ground object of Leifer and Milner [2000].

Note 2. In a logic T, a subobject $U \rightarrowtail A$ is a *proposition* about a type; but U is also an object of T, hence a *type*. This is **comprehension**, or subset types.

$$\frac{x : A \vdash \varphi \text{ prop}}{\{x : A \,|\, \varphi\} \text{ type}}$$

It lets us define *partial operations* $p_{op} : \{\varphi\} \to C$, which we use in §3 to restrict replication to input-guarded processes. We do not use the corresponding refined-binding operations $r_{op} : [\{\varphi\} \to C] \to D$.

2.2 Metatheories and free extension

We will need, in §8, to say that an encoding of a data type is *canonical*. Canonicity will come from initiality, so we record the relevant machinery here.

Every finite limit category is a model of a meta-theory $\mathbb{T}_\wedge$ in Set, presented by the judgements T_0 type, T_1 type, source and target, identities, composition, unitality, associativity, a terminal object and pullbacks; adding types $[A \to B]$ with evaluation and currying gives the meta-theory $\mathbb{T}_\wedge^\to$ of cartesian closed categories. The full presentations are in Appendix B.

Proposition 4. $\mathrm{FinLim} \simeq \mathbb{C}\mathrm{at}(\mathbb{T}_\wedge, \mathrm{Set})$*, and finite-limit-preserving functors correspond to morphisms of models.*

There are analogous meta-theories for each fragment of higher-order predicate logic, connected by inclusions; and the *free model* with respect to such an inclusion is the left Kan extension along it [Barr and Wells, 2013, Section 4.4].

Proposition 5. *Given an equational theory* $\mathrm{T} : \mathbb{T}_\wedge \to \mathrm{Set}$*, the free regular theory is given by the coend*

$$i_\exists!(\mathrm{T})(T) = \vec{\Sigma} S : \mathbb{T}_\wedge .\ \mathrm{T}(S) \times \mathbb{T}_{\wedge\exists}(i_\exists(S), T)$$

and the same holds for every inclusion in the chain

$$\mathbb{T}_\wedge \subset \mathbb{T}_{\wedge\exists} \subset \mathbb{T}_{\wedge\exists\vee} \subset \mathbb{T}_{\wedge\exists\vee\forall} \subset \mathbb{T}_{\wedge\exists\vee\forall\bigvee} \subset \mathbb{T}_{\wedge\exists\vee\forall\bigvee\bigwedge}$$

and their closed variants.

A term of T in the free theory is a pair of a term $s : S$ in the original theory and an inference rule γ from S to T, quotiented by $\langle f(s), \gamma \rangle \equiv \langle s, \gamma(f) \rangle$. These adjunctions induce monads $(-)_\exists$ and so on. We will define bisimilarity in the free infinitary theory $\mathrm{T}_{\wedge\exists\vee\forall\bigvee\bigwedge}$, and in §8 we will use initiality in $\mathbb{T}_\wedge$ to pin down data-type encodings.

3 The theories we treat

Our main theory is the ρ-**calculus**: the π-calculus with reflection between code and data, that is, between processes and names. Its basic rule is communication — an output process synchronizes with an input process along a name, transferring a list of processes which are converted into names and substituted into the input context — together with a rule for dereferencing a quoted process.

ρ calculus

$$\begin{array}{rcl}
 & \vdash & 0 : Pr \\
p_1 : Pr, p_2 : Pr & \vdash & p_1 \,|\, p_2 : Pr \\
p : Pr & \vdash & @(p) : Nm \\
n : Nm & \vdash & *(n) : Pr \\
n : Nm, \vec{p} : Pr^k & \vdash & \mathtt{out}_k(n, \vec{p}) : Pr \\
n : Nm, \lambda\vec{x}.q : [Nm^k \backslash Pr] & \vdash & \mathtt{in}_k(n, \lambda x.q) : Pr
\end{array}$$

$$\begin{array}{rcl}
p : Pr & \vdash & p \,|\, 0 = p \\
p_1 : Pr, p_2 : Pr & \vdash & p_1 \,|\, p_2 = p_2 \,|\, p_1 \\
p_1 : Pr, p_2 : Pr, p_3 : Pr & \vdash & (p_1 \,|\, p_2) \,|\, p_3 = p_1 \,|\, (p_2 \,|\, p_3) \\
n : Nm & \vdash & @(*(n)) = n
\end{array}$$

$$
\begin{array}{rcl}
p_1 : Pr, p_2 : Pr & \vdash & (p_1 \rightsquigarrow p_2)\ \mathrm{prop} \\
n : Nm, \vec{p} : Pr^k, \lambda\vec{x}.q : [Nm^k.Pr] \mid \top & \vdash & \mathtt{out}_k(n, \vec{p}) \mid \mathtt{in}_k(n, \lambda\vec{x}.q) \rightsquigarrow q[@\vec{p}/\vec{x}] \\
p : Pr \mid \top & \vdash & *(@(p)) \rightsquigarrow p \\
p_1, p_2, q : Pr \mid (p_1 \rightsquigarrow p_2) & \vdash & p_1 \mid q \rightsquigarrow p_2 \mid q \\
p_1, p_2, q_1, q_2 : Pr \mid (p_1 \rightsquigarrow p_2), (q_1 \rightsquigarrow q_2) & \vdash & p_1 \mid q_1 \rightsquigarrow p_2 \mid q_2
\end{array}
$$

These rewrites are drawn as string diagrams in Appendix C. Note that $*(@(p)) \rightsquigarrow p$ is a rewrite whose redex is not of the form $t \mid u$; this is the one case the enumeration of Theorem 30 must handle separately.

Following Meredith and Radestock [2005], the calculus above has no name restriction: fresh names are obtained by quotation. Replication, however, we treat as an extension, because it is the smallest interesting instance of the pattern this paper is about — an extension whose defining rewrite is compiled away.

$\rho^!$: ρ + replicated input

$$
\begin{array}{rl}
 & (p, n, c) : \{p : Pr, n : Nm, c : [Nm \backslash Pr] \mid p = \mathtt{in}(n, c)\} \\
\vdash & !(p) : Pr, \\
 & !(p) \rightsquigarrow p \mid !(p)
\end{array}
$$

The comprehension of Note 2 is what lets us restrict ! to input-guarded processes.

π calculus (with replicated input)

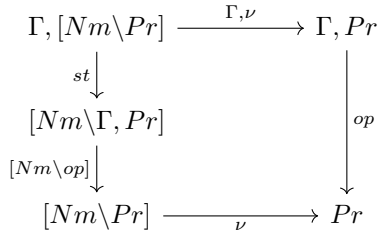
$$
\begin{array}{rcl}
 & \vdash & 0 : Pr \\
n \in \mathrm{Names} & \vdash & n : Nm \\
p_1 : Pr, p_2 : Pr & \vdash & p_1 \mid p_2 : Pr \\
n : Nm, k : Pr & \vdash & \mathtt{out}(n, k) : Pr \\
n : Nm, \lambda x.q : [Nm \backslash Pr] & \vdash & \mathtt{in}(n, \lambda x.q) : Pr \\
\lambda x.q : [Nm \backslash Pr] & \vdash & \nu(\lambda x.q) : Pr \\
p : Pr, n : Nm, c : [Nm \backslash Pr] \mid p = \mathtt{in}(n, c) & \vdash & !(p) : Pr
\end{array}
$$

$$
\begin{array}{rcl}
p : Pr & \vdash & p \mid 0 = p \\
p_1 : Pr, p_2 : Pr & \vdash & p_1 \mid p_2 = p_2 \mid p_1 \\
p_1 : Pr, p_2 : Pr, p_3 : Pr & \vdash & (p_1 \mid p_2) \mid p_3 = p_1 \mid (p_2 \mid p_3) \\
 & \vdash & \nu(\lambda x.0) = 0 \\
p : Pr, \lambda x.q : [Nm \backslash Pr] & \vdash & p \mid \nu(\lambda x.q) = \nu(\lambda x.(p \mid q)) \\
\lambda xy.p : [Nm, Nm \backslash Pr] & \vdash & \nu(\lambda x.\nu(\lambda y.p)) = \nu(\lambda y.\nu(\lambda x.p))
\end{array}
$$

$$
\begin{array}{rcl}
p_1 : Pr, p_2 : Pr & \vdash & (p_1 \rightsquigarrow p_2)\ \mathrm{prop} \\
n : Nm, k : Nm, \lambda x.q : [Nm \backslash Pr] \mid \top & \vdash & \mathtt{out}(n, k) \mid \mathtt{in}(n, \lambda x.q) \rightsquigarrow q[k/x] \\
p_1, p_2, q : Pr \mid (p_1 \rightsquigarrow p_2) & \vdash & p_1 \mid q \rightsquigarrow p_2 \mid q \\
\lambda x.s, \lambda x.t : [Nm \backslash Pr] \mid (\lambda x.s\ (\top_{Nm} \Rightarrow \rightsquigarrow)\ \lambda x.t) & \vdash & \nu(\lambda x.s) \rightsquigarrow \nu(\lambda x.t) \\
p : Pr & \vdash & !(p) \rightsquigarrow p \mid !(p)
\end{array}
$$

The three ν-equations are the ones that an encoding into ρ will satisfy only up to bisimilarity, and they are the reason §5.9 is needed. The equations involve the strength of $[Nm \to -]$: for *op*

any of the unary process operations,



expressing $op(x, \nu(\lambda n.p)) = \nu(\lambda n.op(x, p))$; the commutation of ν with itself uses the symmetry $\sigma : A \times B \cong B \times A$.

Remark 6 (why the ambient calculus is not here). The ambient calculus [Bonchi et al., 2009] is the natural next example and we do not treat it. The reason is structural rather than incidental. Theorem 30 enumerates redex IPOs under the hypothesis that every unary rewrite constructor commutes with parallel composition. For ambients the constructor $n[-]$ does not:

$$n[p \mid q] \neq n[p] \mid q$$

Locality is exactly the failure of that commutation — it is what makes ambients ambients. Consequently the enumeration does not apply, and separately, Bonchi et al. [2009] show that the IPO-derived semantics for ambients is strictly finer than the intended one and has to be coarsened by barbs. Both facts point to a different construction, which we do not attempt here.

4 Admissible observers

Before deriving a transition system we must say which contexts are permitted to observe. This section shows that the answer is not an extra piece of syntax to be legislated, but is determined by a choice of equivalence at the ground types.

4.1 Observation relations

Definition 7. *Let* T *be a lambda theory. An* ***observation relation*** *on* T *is a family* $\approx_A$ *of equivalence relations, one for each type* A *of* T*, subject to the clauses*

$$\begin{aligned} f \approx_{A\times B} g \quad &\text{iff} \quad \pi_1 f \approx_A \pi_1 g \text{ and } \pi_2 f \approx_B \pi_2 g \\ f \approx_{[A\backslash B]} g \quad &\text{iff} \quad a \approx_A a' \text{ implies } f(a) \approx_B g(a') \\ p \approx_{Pr} q \quad &\text{iff} \quad p \text{ and } q \text{ are bisimilar in the derived transition system} \end{aligned}$$

and otherwise freely chosen at the remaining base types. We write $\approx$ *for the whole family and drop the subscript when the type is clear.*

The clauses for products and function types are those of a logical relation; the content of an observation relation is entirely in the choice made at the base types other than Pr.

Definition 8. *An operation* $op : \Gamma \to A$ *of* T *is* ***admissible*** *for* $\approx$ *if it preserves the relation:* $\gamma \approx_\Gamma \gamma'$ *implies* $op(\gamma) \approx_A op(\gamma')$*. We write* $\mathcal{D}(\approx)$ *for the subcategory of* T *generated by the admissible operations, and call its morphisms the* ***observers****.*

Proposition 9. $\mathcal{D}(\approx)$ *contains all identities and projections and is closed under composition; and* $\approx$ *is a congruence for every context in* $\mathcal{D}(\approx)$*.*

Proof. Identities and projections preserve $\approx$ by the product clause. If op and op' preserve $\approx$ then so does $op \circ op'$, since preservation is a closure condition on relations composed forwards. A context $c : \langle\!\langle \vec{Pr} \rangle\!\rangle \to Pr$ in $\mathcal{D}(\approx)$ is by definition a composite of admissible operations, so preserves $\approx$; that is exactly the statement that $\approx$ is a congruence for c. □

Proposition 9 looks tautological and in a sense it is: we have arranged for congruence to hold by defining the observers to be the operations for which it holds. The content is not in the proposition but in the fact that the definition of $\approx_{Pr}$ refers to the derived transition system, which in turn refers to $\mathcal{D}$. The two are mutually recursive, and we must check the recursion has a solution.

Proposition 10. *Fix a choice of $\approx_A$ at each base type other than Pr. Let Φ send a pair $(\mathcal{D}, \approx)$ to $(\mathcal{D}(\approx'), \approx')$, where $\approx'$ is bisimilarity for the $\mathcal{D}$-relative transition system of Definition 20. Then Φ is monotone in the inclusion order on pairs, and so has a greatest fixed point.*

Proof. Enlarging $\mathcal{D}$ admits more labels, hence more transitions to be matched, hence a *finer* bisimilarity: $\mathcal{D} \mapsto \approx$ is antitone. Coarsening $\approx$ makes preservation harder to satisfy, hence admits fewer operations: $\approx \mapsto \mathcal{D}(\approx)$ is antitone. The composite of two antitone maps is monotone, and the pairs form a complete lattice, so Knaster–Tarski applies. □

We take the greatest fixed point throughout: the coarsest observation relation compatible with the largest class of observers preserving it. This is the sense in which labels are compared coinductively rather than syntactically — matching a label c against a label c' asks that $c \approx c'$, with $\approx$ the relation being defined.

4.2 The ρ-calculus: names are discrete

For the ρ-calculus we make the only reasonable choice at the base type Nm.

Definition 11. *The **standard observation relation** on ρ takes $\approx_{Nm}$ to be equality.*

The justification is operational: the communication rewrite fires when two names are equal, and no coarser relation on names is respected by it. A calculus in which channels were matched up to bisimilarity of their quoted processes would have to search an equivalence class at every rendezvous, which is not an implementable rule.

This single choice determines the admissible operations, and in particular it rules one out.

Proposition 12. *For the standard observation relation on ρ, the operation $@ : Pr \to Nm$ is not admissible.*

Proof. Admissibility of @ would require $p \approx_{Pr} q$ to imply $@(p) \approx_{Nm} @(q)$, that is $@(p) = @(q)$, that is $p = q$ by injectivity of quotation. So @ is admissible only if bisimilarity is equality. It is not: take

$$p = \mathtt{out}(a, 0) \mid \mathtt{in}(b, \lambda x.0) \qquad q = \mathtt{in}(b, \lambda x.0) \mid \mathtt{out}(a, 0)$$

which are distinct terms only if we do not quotient by the commutativity equation; for a genuine separation take instead any $p \rightsquigarrow^* q$ with $p \neq q$, for instance $p = *(@(0))$ and $q = 0$, which are weakly bisimilar and not equal. Then $\mathtt{out}(@(p), z)$ and $\mathtt{out}(@(q), z)$ are not bisimilar, since they output on distinct channels and no context can match one action with the other. □

Corollary 13. *For ρ with the standard observation relation, the admissible operations include $\mathtt{out}(n, -)$, $\mathtt{in}(n, -)$, $- \mid -$ and $*(@(-))$, and exclude every operation with a hole under @ alone, in particular $\mathtt{out}(@(-), z)$ and $\mathtt{in}(@(-), z)$.*

Note that $*(@(-))$ is admissible although @ is not: the composite lands back in Pr, where the relation is bisimilarity, and $*(@(p)) \rightsquigarrow p$. Admissibility is a property of composites, not of each factor — which is precisely why $\mathcal{D}(\approx)$ must be described as the subcategory *generated by* admissible operations, and why decomposition-closure (§5) is a real hypothesis rather than a formality.

4.3 Encodings: the observation relation transported

The second source of observation relations is an encoding. If $[\![-]\!] : \mathrm{S} \to \mathrm{T}$ maps types of S to types of T, then the observation relation of S can be transported forwards.

Definition 14. *Let $[\![-]\!] : \mathrm{S} \to \mathrm{T}$ send Pr_{S} to $\langle Pr_{\mathrm{T}} \rangle$ for a meta-context $\langle - \rangle$. The **transported** observation relation $[\![\approx]\!]$ on the image is generated by*

$$[\![p]\!]\ [\![\approx_A]\!]\ [\![q]\!] \quad \textit{iff} \quad p \approx^{\mathrm{S}}_A q$$

and its admissible operations are the images $[\![op]\!]$ of the operations of S*. We write $[\![\mathcal{D}]\!]$ for the subcategory they generate — the image of the source contexts.*

This is the formal content of the slogan that an encoding may only be probed by encoded contexts. It is not a restriction imposed from outside: it is what Definition 7 yields when the relation at the base types is the one the source theory already had. The remainder of the paper is the study of $\mathcal{D}$-relative behavior for these two families of examples.

5 Relative pushouts, relatively

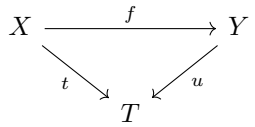
Beginning with Milner [1982], process behavior has been specified by a labelled transition system: a relation $\twoheadrightarrow \subset T \times A \times T$ for which $t \overset{a}{\twoheadrightarrow} u$ means that when t makes the action a it becomes u. This converts internal term structure into external graph structure, revealing how a program can act in different environments.

$$\mathtt{out}(n, p) \xrightarrow{-\,|\,\mathtt{in}(n, \lambda x.c)} c(@p)$$

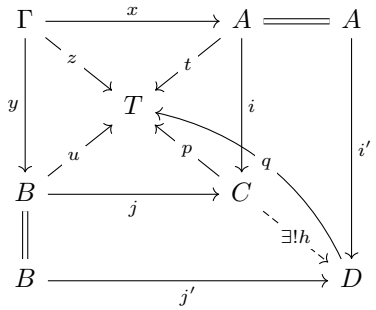
Such systems had to be made by hand, relying on the researcher to know which labels give the correct view of behavior. Leifer and Milner [2000] showed how to derive them: labels are the minimal contexts enabling a rewrite, minimality being a universal property.

5.1 Relative pushouts

Definition 15. *Let* C *be a category and T an object. In the **slice category** C/T an object is a morphism $t : X \to T$ and a morphism $f : t \to u$ is a commuting triangle $t = u \circ f$.*



*A **relative pushout** (RPO) in* C *is a pushout in some C/T: given $x : z \to t$ and $y : z \to u$, the pair p with $i : t \to p$ and $j : u \to p$ is an RPO if for every other candidate (q, i', j') there is a unique $h : p \to q$ with $i' = hi$ and $j' = hj$.*



*An RPO is **idempotent** (an IPO) if p is an identity.*

Because morphisms of a slice category are factorizations, an RPO is the *greatest common factor* of an equal pair of substitutions $t[f] = u[g]$; a transition is asserted when the redex and the context are *relatively prime*.

5.2 Two things Leifer and Milner did not need

The construction of Leifer and Milner [2000] is stated for *reactive systems*, which carry two restrictions we must examine.

First, a reactive system has a distinguished object 0 that is the domain of all IPOs, which in effect restricts the theory to ground terms. Their proofs do not use this object. We therefore adopt an *open* form of the construction, on arbitrary contexts Γ, and Note 1 of §2 is what makes this coherent: currying an entailment produces a rewrite of open processes, so there is no need to close a term before observing it.

Second, a reactive system carries a subcategory $\mathcal{D}$ of *reactive* contexts — those in which a rewrite may occur — closed under composition and decomposition. Their proofs do use this, and they use nothing about the ambient category beyond it. This is the observation we build on: their arguments take place entirely inside $\mathcal{D}$, so they may be run in $\mathcal{D}$. What we change is which restriction $\mathcal{D}$ expresses. Leifer and Milner restrict where reaction may happen; we additionally restrict who may look.

Definition 16. *Let* T *be a lambda theory and* $\mathcal{D} \subseteq \mathrm{T}$ *a subcategory of observers. A* ***$\mathcal{D}$-relative pushout*** *for* $x : z \to t$, $y : z \to u$ *is an RPO computed in the slice* $\mathcal{D}/Pr$*: all four legs and the mediating morphism* h *are required to lie in* $\mathcal{D}$*. A* ***$\mathcal{D}$-relative IPO*** *is one whose* $\mathcal{D}$*-RPO is an identity.*

The point of the relativization is that $\mathcal{D}$-relative IPOs and IPOs are genuinely different, and it is the relative notion that is wanted.

Example 17. Let $[\![-]\!] : \pi \to \rho$ be the encoding of §7, and consider a transition out of $[\![p]\!]$. The ρ-minimal context enabling a communication is a bare $- \mid \mathtt{out}(a, q)$. No such context is of the form $[\![c]\!]$: every encoded context carries the name-supply parameters (n, v) and the replication server $!N$, which ρ-minimality strips away. Insisting on minimality in ρ therefore forces observation from outside the image; insisting on encoded contexts abandons minimality. Relative minimality asks for the smallest *encoded* context, and keeps both.

Definition 18. *Let* T *be a lambda theory and* $\mathcal{D}$ *a subcategory of observers.* T ***has all $\mathcal{D}$-redex-RPOs*** *if for every redex* dl*, seen as a morphism* $l : dl \to d$ *in* $\mathcal{D}/Pr$*, the* $\mathcal{D}$*-relative pushout along* l *exists for all* $t : dl \to c$ *with* $c \in \mathcal{D}$*.*

5.3 Which contexts are observations

We can now finish a definition that earlier drafts of this material left open. The question "which contexts are observations?" has no absolute answer; it has an answer relative to $\approx$.

Definition 19. *Let* $\approx$ *be an observation relation on* T*. A context* $c : \langle\!\langle \vec{Pr} \rangle\!\rangle_1 \to \langle\!\langle \vec{Pr} \rangle\!\rangle_2$ *is an* ***observation*** *if* $c \in \mathcal{D}(\approx)$ *and* $\mathcal{D}(\approx)$ *is closed under decomposition up to* $\approx$*: whenever* $c = c_1 \circ c_2$ *in* T*, there are* $d_1, d_2 \in \mathcal{D}(\approx)$ *with* $d_1 \approx c_1$*,* $d_2 \approx c_2$ *and* $d_1 \circ d_2 \approx c$*.*

Decomposition-closure is what Leifer and Milner [2000] require of their reactive contexts, and it is needed for the same reason: the congruence argument factors a context and must know that the factors are still legitimate. Requiring it only up to $\approx$ rather than on the nose is forced on us by the examples — see Obligation 28.

5.4 The derived transition system

Recall that rewrite operations have the form $op\colon \prod\langle Pr\rangle_i \to Pr$, so their composites have the form $d\colon \langle\!\langle \vec{Pr} \rangle\!\rangle \to Pr$. We assume these closed under decomposition, so that every factorization of d is $f\langle\!\langle \vec{e} \rangle\!\rangle$ with f a rewrite constructor and $\vec{e}$ a tree of rewrite constructors; hence any first factor is of the form $\langle\!\langle \vec{d} \rangle\!\rangle$.

Definition 20. *Let* T *be a lambda theory with all $\mathcal{D}$-redex-RPOs. The* ***$\mathcal{D}$-relative derived transition system*** $\mathrm{T}[\twoheadrightarrow_{\mathcal{D}}]$ *is defined as follows. For each meta-context $\langle\!\langle - \rangle\!\rangle$, each tree of rewrite constructors $\vec{d}$ of type $\langle\!\langle \vec{Pr} \rangle\!\rangle$, and each substitution of base rewrites $[[\Gamma_i \vdash l_i \rightsquigarrow r_i]_j \cdots]_z$, then for each $\mathcal{D}$-relative IPO of the form below with c an observation, assert $\Gamma \vdash \vec{t} \overset{c}{\twoheadrightarrow}_{\mathcal{D}} \langle\!\langle d\langle\!\langle \vec{\ } \rangle\!\rangle \rangle\!\rangle$.*

$$\begin{array}{ccc}
\langle\!\langle \langle\!\langle \vec{\ } \rangle\!\rangle \rangle\!\rangle & \xrightarrow{\ \vec{t}\ } & \langle\!\langle \vec{Pr} \rangle\!\rangle_0 \\
\Big\downarrow{\scriptstyle \langle\!\langle \langle\!\langle \vec{\ } \rangle\!\rangle \rangle\!\rangle} & & \Big\downarrow{\scriptstyle c} \\
\langle\!\langle \langle\!\langle \vec{P} \rangle\!\rangle \rangle\!\rangle & \xrightarrow[\langle\!\langle \vec{d} \rangle\!\rangle]{} & \langle\!\langle \vec{Pr} \rangle\!\rangle
\end{array}$$

Additionally, for each substitution $x : 1 \to \langle\!\langle \langle\!\langle \vec{\ } \rangle\!\rangle \rangle\!\rangle$ into the above diagram, assert $\cdot \vdash \vec{t}x \overset{c}{\twoheadrightarrow}_{\mathcal{D}} \langle\!\langle d\langle\!\langle \vec{\ } \rangle\!\rangle \rangle\!\rangle x$.

Definition 21. *A* ***$\mathcal{D}$-bisimulation*** *is a relation $\sim_{\mathcal{D}} \subset Pr \times Pr$ such that $p_0 \sim_{\mathcal{D}} q_0$ implies that every $p_0 \overset{c}{\twoheadrightarrow}_{\mathcal{D}} p_1$ is matched by some $q_0 \overset{c'}{\twoheadrightarrow}_{\mathcal{D}} q_1$ with $c \approx c'$ and $p_1 \sim_{\mathcal{D}} q_1$, and symmetrically. The* ***weakening*** *$\twoheadrightarrow_{\mathcal{D}*}$ is*

$$p \overset{c}{\twoheadrightarrow}_{\mathcal{D}*} q \quad := \quad \exists \hat{p}, \hat{q}.\ (p \rightsquigarrow^* \hat{p}) \wedge (\hat{p} \overset{c}{\twoheadrightarrow}_{\mathcal{D}} \hat{q}) \wedge (\hat{q} \rightsquigarrow^* q)$$

and ***weak $\mathcal{D}$-bisimilarity*** *$\approx_{\mathcal{D}}$ is defined in the free infinitary theory of Proposition 5 by*

$$\begin{array}{rcl}
(p \approx^0_{\mathcal{D}} q) & := & \top \\
(p \approx^{n+1}_{\mathcal{D}} q) & := & \forall p'.\forall c.(p \overset{c}{\twoheadrightarrow}_{\mathcal{D}} p') \Rightarrow \exists q', c'.(c \approx c') \wedge (q \overset{c'}{\twoheadrightarrow}_{\mathcal{D}*} q') \wedge (p' \approx^n_{\mathcal{D}} q') \wedge \\
 & & \forall q'.\forall c.(q \overset{c}{\twoheadrightarrow}_{\mathcal{D}} q') \Rightarrow \exists p', c'.(c \approx c') \wedge (p \overset{c'}{\twoheadrightarrow}_{\mathcal{D}*} p') \wedge (p' \approx^n_{\mathcal{D}} q') \\
(p \approx_{\mathcal{D}} q) & := & \bigwedge_{i\,:\,\mathbb{N}} p \approx^i_{\mathcal{D}} q
\end{array}$$

At types other than Pr, $\approx_{\mathcal{D}}$ is the observation relation of Definition 7; this is what gives meaning to $\Gamma \vdash t \approx_{\mathcal{D}} u$ for open terms and to $\lambda x.c \approx_{\mathcal{D}} \lambda x.d$ at $[Nm \backslash Pr]$.

Matching labels up to $\approx$ rather than by syntactic identity is not a refinement we could avoid. Two encoded contexts that differ only in the internal name of a fresh channel must be allowed to match, and they are equal only up to bisimilarity.

5.5 Normality and substitution

Definition 22. T *is* ***rw-normal*** *if every ground rewrite is of the form $e \circ x : 1 \to \Gamma \to Rw$ for a generic rewrite e, and* ***eq-normal*** *if every ground equation is $lx = rx$ for an equation (l, r) derived from the presentation.* T *is* ***normal*** *if both hold.*

Normality is what lets us pass between generic and specific rewrites, and it is doing a job that in the wider literature is done by moving to a 2-categorical setting; see Remark 32. Henceforth we assume all theories normal.

Proposition 23. *Suppose $\Gamma \vdash \vec{t} \approx_{\mathcal{D}} \vec{u}$. Then for any $x : 1 \to \Gamma$, $\cdot \vdash \vec{t}x \approx_{\mathcal{D}} \vec{u}x$.*

Proof. Let $\cdot \vdash \vec{t}x \xrightarrow{c}_{\mathcal{D}} z$. By eq-normality $c\vec{t}x = \langle\!\langle d\langle\!\langle \vec{\ } \rangle\!\rangle \rangle\!\rangle x$, and by rw-normality $z = \langle\!\langle d\langle\!\langle \vec{\ } \rangle\!\rangle \rangle\!\rangle x$, so the transition is determined by the diagram

$$\begin{array}{ccccc} 1 & \xrightarrow{\quad x \quad} & \langle\!\langle \langle\!\langle \vec{\ } \rangle\!\rangle \rangle\!\rangle & \xrightarrow{\quad \vec{t} \quad} & \langle\!\langle \vec{P}r \rangle\!\rangle_0 \\ & & \Big\downarrow{\scriptstyle \langle\!\langle \langle\!\langle \vec{\ } \rangle\!\rangle \rangle\!\rangle} & & \Big\downarrow{\scriptstyle c} \\ & & \langle\!\langle \langle\!\langle \vec{P} \rangle\!\rangle \rangle\!\rangle & \xrightarrow[\langle\!\langle \vec{d} \rangle\!\rangle]{} & \langle\!\langle \vec{P}r \rangle\!\rangle \end{array}$$

Because $\vec{t} \approx_{\mathcal{D}} \vec{u}$ there are $\vec{u} \rightsquigarrow^* u' \xrightarrow{c'}_{\mathcal{D}} v' \rightsquigarrow^* z'$ with $c \approx c'$ and $z' \approx_{\mathcal{D}} z$, so there is a $\mathcal{D}$-IPO for u' matching the one above; substituting into $(\rightsquigarrow)$ gives $\vec{u}x \rightsquigarrow^* u'x \xrightarrow{c'}_{\mathcal{D}} v'x \rightsquigarrow^* z'x$. Hence $\{(fx, gx) \mid f \approx_{\mathcal{D}} g\}$ is a $\mathcal{D}$-bisimulation. □

5.6 Congruence

Lemma 24 (Leifer and Milner, 2000, Proposition 2, relativized)**.** *Suppose $\mathcal{D}$ is closed under composition and decomposition up to $\approx$. If the outer rectangle of a pasting is a $\mathcal{D}$-IPO and the left square is a $\mathcal{D}$-RPO, then the right square is a $\mathcal{D}$-IPO.*

Proof. The statement and proof of Leifer and Milner [2000] concern only the slice category in which the pushouts are computed, and use no property of the ambient category. Taking that slice to be $\mathcal{D}/Pr$, which is a category by closure under composition, the argument goes through verbatim; decomposition-closure up to $\approx$ is what guarantees that the factors produced in the course of the argument again lie in $\mathcal{D}$, up to the relation along which labels are matched. □

Theorem 25 ($\mathcal{D}$-relative congruence)**.** *Let* T *have all $\mathcal{D}$-redex-RPOs, with $\mathcal{D}$ closed under composition and decomposition up to $\approx$. Then $\mathcal{D}$-relative strong bisimilarity is a congruence for $\mathcal{D}$: if $\Gamma \vdash t \sim_{\mathcal{D}} u$ then $\Gamma \vdash kt \sim_{\mathcal{D}} ku$ for every $k \in \mathcal{D}$.*

Proof. We show $S = \{(kt, ku) \mid t \sim_{\mathcal{D}} u,\ k \in \mathcal{D}\}$ is a $\mathcal{D}$-bisimulation. Let $kt \xrightarrow{c}_{\mathcal{D}} d\langle\!\langle \vec{r} \rangle\!\rangle$ with $c \in \mathcal{D}$; then the rectangle $c \circ (k, \Delta) = d \circ \langle\!\langle \vec{l} \rangle\!\rangle$ is a $\mathcal{D}$-IPO. Since T has all $\mathcal{D}$-redex-RPOs, the $\mathcal{D}$-RPO of (t, Δ) against $\langle\!\langle \vec{l} \rangle\!\rangle$ exists; it is the left square, with legs c' and d' in $\mathcal{D}$, and by Lemma 24 the right square is a $\mathcal{D}$-IPO. The left square being a $\mathcal{D}$-IPO gives a transition $t \xrightarrow{c'}_{\mathcal{D}} d'\langle\!\langle \vec{r} \rangle\!\rangle$.

Because $t \sim_{\mathcal{D}} u$ there is $u \xrightarrow{c''}_{\mathcal{D}} e'\langle\!\langle \vec{r'} \rangle\!\rangle$ with $c' \approx c''$ and $e'\langle\!\langle \vec{r'} \rangle\!\rangle \sim_{\mathcal{D}} d'\langle\!\langle \vec{r} \rangle\!\rangle$. This determines a second left $\mathcal{D}$-IPO with the same right-hand leg up to $\approx$; pasting it with the right square, which is a $\mathcal{D}$-IPO and lies in $\mathcal{D}$, yields $ku \xrightarrow{c}_{\mathcal{D}} ie'\langle\!\langle \vec{r'} \rangle\!\rangle$ where i is the mediating morphism. Since $kt \xrightarrow{c}_{\mathcal{D}} d\langle\!\langle \vec{r} \rangle\!\rangle = id'\langle\!\langle \vec{r} \rangle\!\rangle$ and $d'\langle\!\langle \vec{r} \rangle\!\rangle \sim_{\mathcal{D}} e'\langle\!\langle \vec{r'} \rangle\!\rangle$, the pair (kt, ku) has matching transitions with successors again in S. □

The shape of this statement is the whole point. It does not say that bisimilarity is a congruence; it says that $\mathcal{D}$-relative bisimilarity is a congruence for $\mathcal{D}$. In §6 this is the difference between a theorem with an embarrassing caveat about reflection and a theorem whose hypothesis names the caveat; in §7 it is the difference between an encoding that fails to preserve behavior and one that preserves it against the observers that matter.

Theorem 26 (weak congruence)**.** OBLIGATION *Under the hypotheses of Theorem 25, together with the condition that every context of $\mathcal{D}$ is either reactive or IPO-uniform in the sense of Di Gianantonio et al. [2009, Section 3], weak $\mathcal{D}$-bisimilarity is a congruence for $\mathcal{D}$.*

We flag this as an obligation rather than a theorem. Weak bisimilarity is not in general a congruence for all contexts, and it is not yet clear to us whether the failure is an artifact of the

framework or a fact of life about weak equivalences; Di Gianantonio et al. [2009] give sufficient conditions in the second-order setting and we expect them to transfer, but we have not verified the transfer. This matters more than it may appear: the results of §7 and §8 are all statements about *weak* bisimilarity, since an encoding replaces a one-step operation by a multi-step protocol. For ρ specifically we give a direct proof in Theorem 36 that does not depend on the general result.

5.7 Existence of relative IPOs

Proposition 27. SKETCH *Let* T *have all redex-RPOs and let* $\mathcal{D}$ *be closed under composition and decomposition up to* $\approx$*. Suppose further that* $\mathcal{D}$ ***reflects factorizations****: whenever* $c \in \mathcal{D}$ *and* $c = f \circ g$ *in* T*, there are* $f', g' \in \mathcal{D}$ *with* $f \approx f'$ *and* $g \approx g'$*. Then* T *has all* $\mathcal{D}$*-redex-RPOs, and the* $\mathcal{D}$*-IPO of a square is the* $\approx$*-least* $\mathcal{D}$*-candidate.*

Proof sketch. Given a redex $\langle\!\langle \vec{l} \rangle\!\rangle$ and t, the $\mathcal{D}$-candidates (c, d) with $c \in \mathcal{D}$ form a preorder under factorization. Reflection of factorizations makes this preorder closed under the greatest common factor computed in T: if (c_1, d_1) and (c_2, d_2) are $\mathcal{D}$-candidates, their T-RPO has legs that factor c_1 and c_2, and by reflection those factors are $\approx$-represented in $\mathcal{D}$. Hence the preorder is codirected and, T having all redex-RPOs, has a least element up to $\approx$; that element is the $\mathcal{D}$-IPO. The gap in this argument is that codirectedness gives a lower bound for each pair but not for an infinite family; a well-foundedness condition on $\mathcal{D}$ is needed and we have not identified the right one. □

Reflection of factorizations is a sharpening of what is elsewhere called *hosting* or faithfulness of an encoding: it asks not merely that source terms embed faithfully, but that the way a source context comes apart is visible in the target. It is exactly the hypothesis the encoding results of §7 consume.

Obligation 28. Verify that $[\![C_\pi]\!] \subseteq \rho$ is decomposition-closed up to $\approx$.

This is the first thing to check and we expect it to fail on the nose. A ρ-factorization of an encoded context can cut a constructed name such as $@(\mathtt{o}(n, 0) \mid \mathtt{i}(n, \lambda x.0))$ into pieces that are not encodings of anything. The question is whether every such factorization is $\approx$-equivalent to one through $[\![C_\pi]\!]$. If the answer is no, the closure of $[\![C_\pi]\!]$ must be computed and shown still to exclude the discriminating contexts of Proposition 12; if the closure swallows them, the approach fails and we would want to know that early.

5.8 Enumerating redex IPOs

Definition 29. *For each rewrite* $\Gamma \vdash l \rightsquigarrow r$*,* l *is a* ***process redex*** *if there is a nontrivial factorization* $l = t(p, \Delta)$ *for* $p : Pr$ *and* $\Delta \subset \Gamma$*. Note that* $*(@(p)) \rightsquigarrow p$ *has no such factorization.*

Theorem 30. SKETCH *Let* $(\mathrm{T}, Pr, \rightsquigarrow)$ *be a lambda theory in which* $(Pr, - \mid -, 0)$ *is a commutative monoid, every basic process redex has the form* $t \mid u$*, the only non-unary rewrite constructor is* $- \mid -$*, and every unary rewrite constructor op commutes with parallel composition:*

$$\begin{array}{ccc} Pr, \langle Pr \rangle & \xrightarrow{Pr, op} & Pr, Pr \\ {\scriptstyle st}\downarrow & & \\ \langle Pr, Pr \rangle & & \downarrow{\scriptstyle - \mid -} \\ {\scriptstyle \langle - \mid - \rangle}\downarrow & & \\ \langle Pr \rangle & \xrightarrow[op]{} & Pr \end{array}$$

Then T *has all redex IPOs, given by the basic factorizations together with one inference rule per*

unary constructor. For each $l \rightsquigarrow r$ and $l = ct$ with $t \neq \mathrm{id}_{Pr}$ we have $\Gamma \vdash t \xrightarrow{c} r$:

$$\begin{array}{ccc} \Gamma & \xrightarrow{t,\Delta} & Pr, \Delta \\ {\scriptstyle l}\downarrow & & \downarrow{\scriptstyle c} \\ Pr & = & Pr \end{array}$$

and for each op with $l \rightsquigarrow r \vdash op\langle l \rangle \rightsquigarrow op\langle r \rangle$, since $l = t \mid u$ and the square above is an IPO, pasting gives

$$\frac{\Gamma \vdash x \xrightarrow{t' \mid u'} r}{\langle \Gamma \rangle \vdash op(x) \xrightarrow{t' \mid u'} op(r)}$$

$$\begin{array}{ccccc} \langle \Gamma \rangle & \xrightarrow{\langle x, \Delta \rangle} & \langle Pr, \Delta \rangle & \xrightarrow{op, Pr} & Pr, Pr \\ {\scriptstyle \langle l \rangle}\downarrow & & \downarrow{\scriptstyle \langle t' \mid u' \rangle} & & \downarrow{\scriptstyle t' \mid u'} \\ \langle Pr \rangle & = & \langle Pr \rangle & \xrightarrow[op]{} & Pr \end{array}$$

Corollary 31. *π satisfies the hypotheses of Theorem 30: its only unary rewrite constructor is ν, and the equation $p \mid \nu(\lambda x.q) = \nu(\lambda x.(p \mid q))$ is exactly the required commutation.*

For ρ the hypotheses hold for the communication rewrite but not for dereferencing, whose redex $*(@(p))$ is not a parallel composition. That rewrite contributes the single additional transition $*(@(p)) \xrightarrow{-} p$, with the identity context as label; the label is an observation because $*(@(-))$ is admissible by Corollary 13, even though @ alone is not. Completing the case analysis for ρ is what makes this theorem a sketch rather than a proof.

Remark 32 (strict pushouts and the 2-categorical alternative). It is well known that strict RPOs frequently fail to exist in the presence of structural congruence, and the standard repair is to move to a bicategorical setting and use *groupoidal* relative pushouts [Sassone and Sobociński, 2003, 2005]. The commutative-monoid equations on $- \mid -$ put us squarely in that territory. Our normality assumption is doing the same work by other means, and we regard the relationship as unresolved: either normality should be justified as a hypothesis a MeTTaIL presentation can be checked against, or the construction should be redone with GRPOs and normality dropped. We note that our matching of labels up to $\approx$ rather than on the nose is already a step in the 2-categorical direction, since it replaces equality of labels by a coherent isomorphism.

5.9 Working up to bisimilarity

There is a second relativization we need, and it turns out to be the same one.

An encoding will in general fail to preserve the equations of its source on the nose. The ν-equations of π are the standard example: $[\![p \mid \nu(\lambda x.q)]\!]$ and $[\![\nu(\lambda x.(p \mid q))]\!]$ are distinct ρ terms that are bisimilar. An earlier approach to this problem replaced the equations of the source by an internal congruence Eq and asked for relative pushouts with respect to it. That construction requires rebuilding the framework and we do not pursue it, because the following suffices.

Definition 33. *Let $\mathcal{D}$ be a subcategory of observers and $\approx$ the observation relation. The category $\mathcal{D}/\approx$ has the objects of $\mathcal{D}$ and $\approx$-classes of morphisms as morphisms. Composition is well defined because $\approx$ is a congruence for $\mathcal{D}$ (Proposition 9).*

Proposition 34. *A functor $\mathrm{S} \to \mathcal{D}/\approx$ is the same thing as a map $[\![-]\!]$ of presentations such that $p =_{\mathrm{S}} q$ implies $[\![p]\!] \approx [\![q]\!]$. Relative pushouts computed in $\mathcal{D}/\approx$ are the $\mathcal{D}$-relative pushouts of Definition 16 with labels matched up to $\approx$.*

Proof. The first claim is the universal property of the quotient: a functor out of S must send equal terms to equal morphisms of $\mathcal{D}/\approx$, and equality there is $\approx$. The second is Definition 33 unwound, since a candidate in the quotient is an $\approx$-class of candidates in $\mathcal{D}$. □

So the single move of working in $\mathcal{D}/\approx$ handles both restrictions at once: the subcategory says which contexts may observe, and the quotient says that they observe only up to bisimilarity. Every encoding result below is stated in $\mathcal{D}/\approx$, and the clause "equations become bisimulations" is not an extra demand on encodings but the definition of a functor into the quotient.

6 First instance: reflection in the ρ-calculus

The ρ-calculus is the case in which the observer subcategory is proper for a reason internal to the theory. We record the structural property that makes its behavior tractable, and then prove congruence for the admissible contexts of Corollary 13.

Definition 35. *A lambda theory is **transparent** if for every context c that is not reactive there is a unique context $\overline{c}$ such that*

$$c(t) \overset{\overline{c}}{\twoheadrightarrow} d(t)$$

where d is constructed from the variables of $\overline{c}$ and does not involve the application of c to t.

Transparency says that a non-reactive context can always be peeled off by exactly one complementary observation. In ρ the pairing is visible immediately:

$$\begin{aligned} \mathtt{out}(n,p) &\xrightarrow{-\,|\,\mathtt{in}(n,c)} c(@p) \\ \mathtt{in}(n,c) &\xrightarrow{-\,|\,\mathtt{out}(n,p)} c(@p) \\ *(@(p)) &\overset{-}{\twoheadrightarrow} p \end{aligned}$$

so that $\overline{\mathtt{out}(n,-)} = -\,|\,\mathtt{in}(n,c)$ and dually, and $*(@(-))$ is its own complement with the identity label. Transparency is what makes the direct argument below possible without appeal to Theorem 25, and hence without depending on the outstanding Obligation 28.

Theorem 36. *In the ρ-calculus with the standard observation relation, weak $\mathcal{D}$-bisimilarity is a congruence for the operations generated by*

$$\begin{aligned} \mathtt{out}(n,-) &\qquad Pr \to Pr \\ \mathtt{in}(n,-) &\qquad [Nm\backslash Pr] \to Pr \\ -\,|\,- &\qquad Pr, Pr \to Pr \\ *(@(-)) &\qquad Pr \to Pr \end{aligned}$$

when observations are restricted to the same class.

Proof. Let $p \approx_{\mathcal{D}} q$ and consider a transition out of $\mathtt{out}(n,p)$. Because p cannot rewrite within $\mathtt{out}$, the only transition is

$$\mathtt{out}(n,p) \xrightarrow{-\,|\,\mathtt{in}(n,c)}_{\mathcal{D}} c(@p)$$

matched by $\mathtt{out}(n,q) \xrightarrow{-\,|\,\mathtt{in}(n,c)}_{\mathcal{D}} c(@q)$; the result follows by structural induction, the successors being related because $\approx_{\mathcal{D}}$ at $[Nm\backslash Pr]$ is the logical relation of Definition 7.

Let $\lambda x.c \approx_{\mathcal{D}} \lambda x.d : [Nm\backslash Pr]$ and consider a transition out of $\mathtt{in}(n, \lambda x.c)$. Because $\lambda x.c$ cannot rewrite within $\mathtt{in}$, the only transition is

$$\mathtt{in}(n, \lambda x.c) \xrightarrow{-\,|\,\mathtt{out}(n,v)}_{\mathcal{D}} c(@v)$$

matched by $\mathtt{in}(n, \lambda x.d) \xrightarrow{-\,|\,\mathtt{out}(n,v)}_{\mathcal{D}} d(@v)$, and $c(@v) \approx_{\mathcal{D}} d(@v)$ by the function-type clause together with Proposition 23.

Let $p_1 \approx_{\mathcal{D}} q_1$ and $p_2 \approx_{\mathcal{D}} q_2$ and consider $p_1 \,|\, p_2 \overset{f}{\twoheadrightarrow}_{\mathcal{D}} t$. If $f = (-)$ then by transparency $p_1 = \mathtt{op}(n,-) \,|\, \hat{p}_1$ and $p_2 = \overline{\mathtt{op}}(n,-) \,|\, \hat{p}_2$, so

$$p_1 \xrightarrow{-\,|\,\overline{\mathtt{op}}(n,-)}_{\mathcal{D}} \hat{p}_1 \quad \text{and} \quad p_2 \xrightarrow{-\,|\,\mathtt{op}(n,-)}_{\mathcal{D}} \hat{p}_2$$

matched by weak transitions

$$q_1 \rightsquigarrow^* u \xrightarrow{\mathtt{op}(n,-)}_{\mathcal{D}} \hat{u} \rightsquigarrow^* \hat{q}_1 \quad \text{and} \quad q_2 \rightsquigarrow^* v \xrightarrow{\overline{\mathtt{op}}(n,-)}_{\mathcal{D}} \hat{v} \rightsquigarrow^* \hat{q}_2$$

and thus $q_1 \,|\, q_2 \rightsquigarrow^* u \,|\, v \overset{-}{\twoheadrightarrow}_{\mathcal{D}} \hat{u} \,|\, \hat{v} \rightsquigarrow^* \hat{q}_1 \,|\, \hat{q}_2$. If $f \neq (-)$ then it is $-\,|\,\mathtt{out}(n,v)$ or $-\,|\,\mathtt{in}(n,c)$ for communication with p_1 or p_2, matched by $\rightsquigarrow^* \xrightarrow{-\,|\,\mathtt{op}}_{\mathcal{D}} \rightsquigarrow^*$ on the corresponding side. So $q_1 \,|\, q_2$ matches every $\rightsquigarrow$ with $\rightsquigarrow^*$ and every $\overset{c}{\twoheadrightarrow}_{\mathcal{D}}$ with $\rightsquigarrow^* \overset{c}{\twoheadrightarrow}_{\mathcal{D}} \rightsquigarrow^*$, giving $p_1 \,|\, p_2 \approx_{\mathcal{D}} q_1 \,|\, q_2$.

Finally $*(@(-))$ preserves $\approx_{\mathcal{D}}$ because $*(@(p)) \rightsquigarrow p$ and $*(@(q)) \rightsquigarrow q$, so any transition of one is matched by the other after a silent step. □

Remark 37. The theorem is stated for the admissible class and is false outside it, by Proposition 12. Earlier drafts recorded this as a caveat — "there is just one problem, reflection" — and then restricted the theorem by hand. In the present framework the restriction is not an afterthought: $\mathcal{D}$ was defined so that @ falls outside it, and the theorem is an instance of the general shape of Theorem 25. The ρ-calculus is not defective; it is a theory whose observer subcategory is proper.

7 Second instance: encodings

7.1 What an encoding must preserve

The naive demand on a translation is that it be a functor of lambda theories preserving bisimilarity on the nose. Written out, this asks that $[\![-]\!]$ preserve products, pullbacks and function types, map Pr_{S} into $\langle Pr_{\mathrm{T}} \rangle$ for a meta-context $\langle - \rangle$, map rewriting to a silent path, and satisfy $p \approx q \vdash [\![p]\!] \approx [\![q]\!]$ where the right-hand $\approx$ is bisimilarity in T.

This demand is not merely hard to meet; it is the wrong demand, and Example 17 says why. The target has observers the source never had. We therefore ask instead for a functor into $\mathcal{D}/\!\approx$, and Proposition 34 tells us what that amounts to.

Definition 38. *Let* S, T *be lambda theories. An* ***encoding*** $[\![-]\!] : \mathrm{S} \to \mathrm{T}$ *is a functor* $\mathrm{S} \to [\![\mathcal{D}]\!]/\!\approx$, *where* $[\![\mathcal{D}]\!]$ *is the transported observer subcategory of Definition 14. Explicitly,* $[\![-]\!]$

- *preserves conjunction and substitution, that is products and pullbacks;*
- *preserves function types;*
- *maps* Pr_{S} *to* $\langle Pr_{\mathrm{T}} \rangle$ *for some meta-context* $\langle - \rangle$*;*
- *sends each equation of* S *to a bisimulation:* $p =_{\mathrm{S}} q$ *implies* $[\![p]\!] \approx [\![q]\!]$*;*
- *maps rewriting to a silent path:* $p \rightsquigarrow q$ *implies* $[\![p]\!](\rightsquigarrow^* \approx)^* [\![q]\!]$*, and is called* ***strong*** *if* $[\![p]\!] \rightsquigarrow^* \sim [\![q]\!]$*;*
- *and preserves behavior against the transported observers:* $p \approx^{\mathrm{S}}_{\mathcal{D}} q$ *iff* $[\![p]\!] \approx_{\mathcal{D}} [\![q]\!]$ *in* $[\![\mathcal{D}]\!]$.

The last clause is stated as a biconditional. Soundness — the forward direction — is what a translation must have to be usable at all. The converse is what makes the restriction to $[\![\mathcal{D}]\!]$ the *right* restriction rather than merely a sufficient one, and it is the direction that fails badly against unrestricted observers, since the target can then separate images of equivalent programs by reading their implementation.

7.2 Eliminating replication

The smallest instance is the elimination of replicated input from $\rho^!$, using a duplicator stored at a name [Lybech, 2022].

Proposition 39 (Lybech, 2022)**.** *There is an encoding of $\rho^!$ into ρ given by*

$$\begin{aligned} m, n : Nm &\vdash m \cdot n := @(\mathtt{o}(m, 0) \mid \mathtt{i}(n, \lambda x.0)) : Nm \\ x : Nm &\vdash D(x) := \mathtt{i}(x, \lambda y.\ *\, y \mid \mathtt{o}(x, *y)) : Pr \end{aligned}$$

$$[\![!\mathtt{in}(a, \lambda x.p)]\!] = \lambda n.D(n) \mid \mathtt{out}(n, \mathtt{in}(a, \lambda x.[\![p]\!](n \cdot n) \mid D(n)))$$

The rewrite $!(p) \rightsquigarrow p \mid !(p)$ maps to a path from $[\![p]\!]$ to a process bisimilar to $[\![p \mid !(p)]\!]$.

$$\begin{aligned} [\![!\mathtt{in}(a, \lambda x.p)]\!](n) &= D(n) \mid \mathtt{out}(n, \mathtt{in}(a, \lambda x.[\![p]\!](n \cdot n) \mid D(n))) \\ &= \mathtt{i}(n, \lambda y.\ast y \mid \mathtt{o}(n, *y)) \mid \mathtt{o}(n, \mathtt{i}(a, \lambda x.[\![p]\!] \mid D(n))) \\ &\rightsquigarrow\ *(@(\mathtt{i}(a, \lambda x.[\![p]\!] \mid D(n)))) \mid \mathtt{o}(n, *(@(\mathtt{i}(a, \lambda x.[\![p]\!] \mid D(n))))) \\ (1) \quad &\rightsquigarrow\ \mathtt{i}(a, \lambda x.[\![p]\!] \mid D(n)) \mid \mathtt{o}(n, *(@(\mathtt{i}(a, \lambda x.[\![p]\!] \mid D(n))))) \\ (2) \quad &\approx\ [\![\mathtt{in}(a, \lambda x.p)]\!] \mid [\![!\mathtt{in}(a, \lambda x.p)]\!] \end{aligned}$$

A copy of $[\![\lambda x.p]\!]$ is stored at a and the same is being output; for any receive on a, the body runs and another duplicator opens, reforming the original redex.

$$\begin{aligned} (1) \quad &\xrightarrow{\mathtt{out}(a,r) \mid -}\ (([\![p]\!][@r] \mid D(n)) \mid \mathtt{out}(n, \mathtt{in}(a, \lambda x.[\![p]\!] \mid D(n)) \\ &=\ [\![p]\!][@r] \mid [\![!\mathtt{in}(a, \lambda x.p)]\!] \\ (2) \quad &\xrightarrow{\mathtt{out}(a,r) \mid -}\ [\![p]\!][@r] \mid [\![!\mathtt{in}(a, \lambda x.p)]\!] \end{aligned}$$

Theorem 40. *If $p \approx_{\mathcal{D}} q$ in $\mathrm{T}^!_\rho$ then $[\![p]\!] \approx_{\mathcal{D}} [\![q]\!]$ in T_ρ.*

Proof. The only difference of $\mathrm{T}^!_\rho$ from T_ρ is the operation $! : \{\mathtt{in}(n, c)\} \to Pr$ and the rewrite $!(p) \rightsquigarrow p \mid !(p)$, with no equations between ! and the other operations; so it has exactly one redex IPO not already in T_ρ. It therefore suffices to consider $\lambda x.p \approx_{\mathcal{D}} \lambda x.q$ and apply $[\![!\mathtt{in}(n, -)]\!]$. Because the result is an abstraction, the first transition must be an evaluation at a particular n.

Suppose $[\![!\mathtt{in}(a, \lambda x.p)]\!](n) \overset{c}{\twoheadrightarrow}_{\mathcal{D}} t$. Then c is $(-)$, $(- \mid \mathtt{out}(n, q))$, or $(- \mid \mathtt{in}(n, c))$. In the latter two cases the rewrites of the duplicator are mirrored in the two processes. If $c = (-)$ the process evolves to (1) above, and the same transitions apply to $[\![!\mathtt{in}(n, \lambda x.q)]\!]$; because $\lambda x.p \approx_{\mathcal{D}} \lambda x.q$, they are bisimilar for every substitution $p[@r] \approx_{\mathcal{D}} q[@r]$ by Proposition 23. Hence $[\![!\mathtt{in}(a, \lambda x.p)]\!] \approx_{\mathcal{D}} [\![!\mathtt{in}(a, \lambda x.q)]\!]$. □

This is the pattern the rest of the paper generalizes: an operation with a rewrite rule is replaced by a protocol, the rewrite becomes a silent path, and the encoding is behavior-preserving against the encoded observers.

7.3 π into ρ

The main encoding uses the same duplicator, plus a name server $!N$ that supplies fresh names to interpret ν.

Theorem 41. OBLIGATION *There is an encoding of the π calculus with replicated input into the ρ calculus, given as follows.*

$$
\begin{array}{rcl}
n : Nm & \vdash & +n := @(\mathtt{o}(n, 0)) : Nm \\
n : Nm & \vdash & n+ := @(\mathtt{i}(n, \lambda x.0)) : Nm \\
m, n : Nm & \vdash & m \cdot n := @(\mathtt{o}(m, 0) \mid \mathtt{i}(n, \lambda x.0)) : Nm \\
x : Nm & \vdash & D(x) := \mathtt{i}(x, \lambda y.\ *y \mid \mathtt{o}(x, *y)) : Pr \\
x, w, v, s : Nm & \vdash & !N(x, w, v, s) := \\
 & & D(x) \mid \mathtt{o}(x, \mathtt{i}(w, \lambda a.\mathtt{i}(v, \lambda r.D(x) \mid \mathtt{o}(r, *a) \mid \mathtt{o}(w, \mathtt{o}(a, 0))))) \mid \mathtt{o}(w, *s)
\end{array}
$$

$$
\begin{array}{rcl}
[\![p]\!] & = & \lambda xzvns.[\![p]\!](n, v) \mid !N(x, z, v, s) \\
[\![0]\!](n, v) & = & 0 \\
[\![p \mid q]\!](n, v) & = & [\![p]\!](+n, v) \mid [\![q]\!](n+, v) \\
[\![\mathtt{out}(a, k)]\!](n, v) & = & \mathtt{out}(a, *(k)) \\
[\![\mathtt{in}(a, \lambda x.p)]\!](n, v) & = & \mathtt{in}(a, \lambda x.[\![p]\!](n, v)) \\
[\![\nu(\lambda x.p)]\!](n, v) & = & \mathtt{out}(v, *(n)) \mid \mathtt{in}(n, \lambda x.[\![p]\!](n \cdot n, v)) \\
[\![!\mathtt{in}(a, \lambda x.p)]\!](n, v) & = & D(n) \mid \mathtt{out}(n, \mathtt{in}(a, \lambda x.D(n) \mid [\![p]\!](n \cdot n, v)))
\end{array}
$$

We state the proof obligations rather than the proof. The argument factors through the two lemmas below, of which the second is Theorem 40.

Obligation 42 (the ν lemma). The three ν-equations of π become bisimulations:

$$
\begin{array}{rcl}
[\![\nu(\lambda x.0)]\!](n, v) & \approx & 0 \\
[\![p \mid \nu(\lambda x.q)]\!](n, v) & \approx & [\![\nu(\lambda x.(p \mid q))]\!](n, v) \\
[\![\nu(\lambda x.\nu(\lambda y.p))]\!](n, v) & \approx & [\![\nu(\lambda y.\nu(\lambda x.p))]\!](n, v)
\end{array}
$$

equivalently, that $[\![-]\!]$ is a functor into $[\![\mathcal{D}]\!]/\approx$ on the ν fragment.

This is the whole difficulty and it is where the effort should go. The name-supply discipline is what carries it: $+n$, $n+$ and $n \cdot n$ generate a binary tree of names from a single root, and the second equation holds because relocating a ν across a parallel composition permutes which subtree a fresh name is drawn from, which no encoded context can detect. That last clause is exactly a $[\![\mathcal{D}]\!]$-relative statement and is false for arbitrary ρ observers, which can send on $+n$ directly.

Remark 43 (staging). We suggest proving Obligation 42 against $\rho^!$ rather than ρ, and composing with Theorem 40:

$$\pi \longrightarrow \rho^! \longrightarrow \rho$$

The second leg is done. The first leg then concerns only ν and the name supply, with replication handled natively. The factorization is not free, since $[\![-]\!]$ into ρ is parameterized by (n, v) where the replication encoding needs only n; but it isolates name creation, which is the part nobody has written down.

Obligation 44. Full abstraction: $[\![p]\!] \approx_{\mathcal{D}} [\![q]\!]$ in $[\![\mathcal{D}]\!]$ implies $p \approx_{\mathcal{D}} q$ in π.

8 Third instance: extension by data types

We come to the motivating case. A user of MeTTaIL presents a pure theory — λ, ρ, π, or a domain-specific calculus of their own — with no built-in data. They then want `Bool`, `Int`, `String`, lists. Adding these adds operations and, crucially, *equations*: $2 + 3 = 5$ is not a rewrite the user writes, it is an identity they expect to hold. The question is whether the reasoning they do in the extended theory survives compilation back into the pure one.

8.1 Extensions and models

Definition 45. *A **finitary algebraic theory** A is a presentation with finitely many sorts, finitely many operations of finite arity, and equations; no rewrites. Its **initial model** is the free model on no generators.*

Definition 46. *Let T be a lambda theory and A a finitary algebraic theory. The **extension** $\mathrm{T} \otimes \mathrm{A}$ is T together with the sorts, operations and equations of A and an injection $\iota_a : a \to Pr$ for each sort a of A. The extension is **pure** if no new equation mentions an operation of T other than the injections, and **conservative** if $\mathrm{T} \to \mathrm{T} \otimes \mathrm{A}$ reflects equations and rewrites between terms of T.*

Purity and conservativity are exactly the conditions a language-definition platform can check mechanically from a presentation, and they are what make the following possible; without purity there is no reason to expect an encoding that is the identity on T.

Definition 47. *A **lax model** M of A in T assigns to each sort a an object $\mathrm{M}(a)$ of T and to each operation f of A a morphism $\mathrm{M}(f)$ of the corresponding type, such that every equation of A holds up to weak $\mathcal{D}$-bisimilarity:*

$$\mathrm{A} \vdash s = t \quad \textit{implies} \quad \mathrm{M}(s) \approx_{\mathcal{D}} \mathrm{M}(t)$$

The word *lax* is the only honest one. A strict model would satisfy the equations on the nose, and no encoding of a data type into a process calculus does: the built-in $2 + 3$ is an identity, taking no steps, while its encoding is a protocol taking several. Laxity is not a weakness of the construction; it is what the construction is about.

8.2 Canonicity

There are many encodings of `Bool` into a calculus — Church, Scott, Parigot, and any number of ad hoc ones. Calling one *canonical* requires saying with respect to what.

Proposition 48. *Let A be finitary algebraic and M a lax model of A in T. Then M determines, up to weak $\mathcal{D}$-bisimilarity, a unique map on the A-terms of $\mathrm{T} \otimes \mathrm{A}$ commuting with the operations. Canonicity is thus canonicity relative to the choice of M, and the encoding of the closed A-terms is determined by initiality.*

Proof. The closed A-terms form the initial model of A; a lax model is a model in the quotient $\mathcal{D}/\!\approx$ by Proposition 34, so there is a unique morphism from the initial model to it, and uniqueness in the quotient is uniqueness up to $\approx$. This is the free-extension machinery of Proposition 5 applied to the inclusion of the empty presentation. □

For the calculi of §3 we take M to be the **Scott-style** model, in which a value of an algebraic sort is a process that, when handed a family of continuation names, signals on the one corresponding to its outermost constructor and delivers its arguments there. In a calculus with pattern matching this is the right choice: destructuring is one communication rather than a fold, which matters for a platform whose surface language has a for-comprehension with patterns. Church-style models are also lax models and everything below applies to them; they are simply worse.

8.3 The name discipline

Scott-style encodings need a supply of private names. In ρ these are generated by quotation, using the operations already introduced in §7:

$$+n := @(\mathtt{o}(n, 0)) \qquad n+ := @(\mathtt{i}(n, \lambda x.0)) \qquad n \cdot n := @(\mathtt{o}(n, 0) \mid \mathtt{i}(n, \lambda x.0))$$

Every encoded term is parameterized by a root name n, and the parallel clause splits the root, $[\![p \mid q]\!](n) = [\![p]\!](+n) \mid [\![q]\!](n+)$, so that occurrences receive distinct roots.

Lemma 49 (name discipline)**.** *Under the root-splitting discipline, distinct occurrences of encoded subterms receive incomparable roots, and the derived names of one occurrence are the derived names of no other.*

Proof. The maps $n \mapsto +n$ and $n \mapsto n+$ are injective with disjoint images, since $\mathtt{o}(n, 0)$ and $\mathtt{i}(n, \lambda x.0)$ are distinct terms and @ is injective. So roots form a binary tree and the derived names of a node lie in its own subtree. □

Lemma 50 (root independence)**.** *For m, n roots not occurring in $[\![p]\!]$, $[\![p]\!](m) \approx_{\mathcal{D}} [\![p]\!](n)$ in $[\![\mathcal{D}]\!]$.*

Proof. The root parameter appears only in derived names used for internal signalling. By Lemma 49 those names lie in the occurrence's own subtree, and by Definition 14 no encoded context addresses them, since the source theory has no operation whose image sends on a derived name of another occurrence. So the relation pairing $[\![p]\!](m)$ with $[\![p]\!](n)$ and closed under the transitions of $[\![\mathcal{D}]\!]$ is a $\mathcal{D}$-bisimulation. □

Lemma 50 is false against unrestricted ρ observers, which can send on $+m$ and not on $+n$. It is the first place where the whole apparatus of §4 earns its keep, and it is not a technicality: without it no encoding of a binder-free data type is well defined.

8.4 Booleans, in full

Let $\mathrm{A}_{\mathtt{Bool}}$ have one sort, constants $\mathtt{tt}, \mathtt{ff}$, an operation $\mathtt{if}(-,-,-)$, and the two equations

$$\mathtt{if}(\mathtt{tt}, x, y) = x \qquad \mathtt{if}(\mathtt{ff}, x, y) = y$$

from which negation, conjunction and disjunction are definable in the usual way. The Scott model in ρ is

$$\begin{aligned} [\![\mathtt{tt}]\!](n) &= \mathtt{out}(+n, 0) \\ [\![\mathtt{ff}]\!](n) &= \mathtt{out}(n+, 0) \\ [\![\mathtt{if}(b, p, q)]\!](n) &= [\![b]\!](n) \mid \mathtt{in}(+n, \lambda y.[\![p]\!](n \cdot n)) \mid \mathtt{in}(n+, \lambda y.[\![q]\!](n \cdot n)) \end{aligned}$$

Lemma 51 (dead branches are inert)**.** *Let n be the root of a conditional whose test has been consumed. Then $\mathtt{in}(n+, \lambda y.[\![q]\!](n \cdot n)) \approx_{\mathcal{D}} 0$ in $[\![\mathcal{D}]\!]$.*

Proof. A transition out of the residual requires an observation sending on $n+$. By Lemma 49 the only encoded producer of a message on $n+$ is $[\![\mathtt{ff}]\!](n)$, and the root n belongs to this occurrence alone; the test has been consumed, so no such producer remains in the term. By Definition 14 no context in $[\![\mathcal{D}]\!]$ supplies one either, since every encoded context addresses names derived from its own root. Hence the residual has no transitions in the $[\![\mathcal{D}]\!]$-relative system, and the singleton relation pairing it with 0 is a $\mathcal{D}$-bisimulation. □

Proposition 52. *The Scott model of $\mathrm{A}_{\mathtt{Bool}}$ in ρ is a lax model: both equations become weak $[\![\mathcal{D}]\!]$-bisimulations.*

Proof. Compute the first; the second is symmetric.

$$\begin{aligned} [\![\mathtt{if}(\mathtt{tt}, p, q)]\!](n) &= \mathtt{out}(+n, 0) \mid \mathtt{in}(+n, \lambda y.[\![p]\!](n \cdot n)) \mid \mathtt{in}(n+, \lambda y.[\![q]\!](n \cdot n)) \\ &\rightsquigarrow [\![p]\!](n \cdot n) \mid \mathtt{in}(n+, \lambda y.[\![q]\!](n \cdot n)) \\ &\approx_{\mathcal{D}} [\![p]\!](n \cdot n) \qquad \text{by Lemma 51} \\ &\approx_{\mathcal{D}} [\![p]\!](n) \qquad \text{by Lemma 50} \end{aligned}$$

The first step is the communication rewrite; it is a silent step, so the composite is a weak bisimulation and not a strong one. □

Remark 53. Every line of that computation after the first is $[\![\mathcal{D}]\!]$-relative and false without the restriction. Against arbitrary ρ observers the residual $\mathtt{in}(n+, \dots)$ is live — a context may send on $n+$ and fire the branch that was not taken — so $[\![\mathtt{if}(\mathtt{tt}, p, q)]\!]$ and $[\![p]\!]$ are separated, and the compiled conditional is simply wrong. This is the clearest statement we know of why the observer subcategory cannot be dispensed with. It is also the standard "junk" problem of encodings, and the $\mathcal{D}$-relative treatment is the standard fix, made into a definition.

8.5 The general theorem

Theorem 54. SKETCH *Let* T *be a lambda theory with all $\mathcal{D}$-redex-RPOs,* A *a finitary algebraic theory,* $\mathrm{T} \otimes \mathrm{A}$ *a pure conservative extension, and* M *a lax model of* A *in* T *whose operations are admissible. Then the induced* $[\![-]\!]_{\mathrm{M}} : \mathrm{T} \otimes \mathrm{A} \to \mathrm{T}$ *is an encoding in the sense of Definition 38: it is the identity on* T*, and every equation of* $\mathrm{T} \otimes \mathrm{A}$ *becomes a weak $\mathcal{D}$-bisimulation.*

Proof sketch. Identity on T is purity: no new equation constrains an old operation, so the old fragment is untouched, and conservativity ensures nothing new is derivable there.

For the equations, induct on the derivation of $s = t$ in $\mathrm{T} \otimes \mathrm{A}$. The axioms of A hold up to $\approx_{\mathcal{D}}$ by Definition 47. Reflexivity, symmetry and transitivity are inherited because $\approx_{\mathcal{D}}$ is an equivalence. Congruence steps — from $s_i = t_i$ conclude $f(\vec{s}) = f(\vec{t})$ — hold because the operations of M are admissible, so $\approx_{\mathcal{D}}$ is a congruence for them by Proposition 9, or in the case of ρ by Theorem 36. Substitution steps hold by Proposition 23. Purity is what guarantees that no derivation mixes an A-axiom with a T-equation in a way not covered by these cases.

Two gaps. First, the induction is over derivations in the free infinitary theory, and the infinitary conjunction defining $\approx_{\mathcal{D}}$ must be shown to commute with the induction; this is where a well-foundedness hypothesis on A is needed and A being finitary is what should supply it. Second, admissibility of the operations of M is a per-model check, and we have carried it out only for $\mathrm{A}_{\mathtt{Bool}}$. □

Obligation 55. Strengthen Theorem 54 to a retraction: exhibit $\mathrm{T} \otimes \mathrm{A} \to \mathrm{T}$ and $\mathrm{T} \to \mathrm{T} \otimes \mathrm{A}$ with the composite $[\![\mathcal{D}]\!]$-relatively bisimilar to the identity, naturally in T. Naturality in T is the formal content of the claim that extensions "just work" for any pure theory a user may present, rather than for each one separately.

8.6 What is out of scope, and why

Remark 56 (`Float`). We scope the theorem to initial algebras of finitary signatures: `Bool`, `Nat` and `Int`, `String` as a free monoid, lists, finite maps. Floating-point arithmetic is not among them and should not be. IEEE 754 addition is not associative, has two zeros, and has a value unequal to itself; there is no algebraic theory whose initial model it is, hence nothing for an encoding to be canonical with respect to. The right treatment is to carry `Float` as an opaque ground type with an oracle, whose observation relation at that type is a chosen equivalence in the sense of Definition 7 — which is precisely the freedom that definition leaves open at base types other than Pr.

Remark 57 (cost). In a cost-accounted theory Theorem 54 is false as stated. The built-in $2+3$ and the encoded $[\![2+3]\!]$ carry different ledgers: the first is an equation and costs nothing, the second is a protocol and costs several communications. No cost-respecting bisimulation relates them.

We regard the gap as the content rather than an obstruction. The encoding theorem says that extension by data types is *semantically* free; cost accounting says that data types are exactly what one pays less for. Together they say why a Turing-complete language acquires an `Int` at all. The two statements should be proved separately, over the uncosted and costed theories respectively, and their difference — the ledger gap of an encoding — is a quantity worth naming.

Remark 58 (what a platform must check). Theorem 54 is stated so that its hypotheses are decidable properties of a presentation. Given a MeTTaIL presentation of T and of an extension, a compiler

can check that the extension is pure (no new equation mentions an old operation), that A is finitary algebraic (no rewrites among the new rules), and that the supplied model's operations are admissible for the declared observation relation. Conservativity and the well-foundedness gap of Theorem 54 are the parts that are not obviously mechanical, and are where we would look first if the guarantee is to be delivered by a tool rather than by hand.

9 Related work

Deriving labels. The framework is that of Leifer and Milner [2000], refined for bigraphs by Milner and Jensen [2004]. Klin et al. [2005] give a general account of the "labels from reductions" programme and its limits. Our departure is to make the observer class a parameter rather than take it to be the ambient category; the technique of restricting to a subcategory is theirs, applied to reaction rather than observation.

Structural congruence and 2-categories. Strict relative pushouts are fragile in the presence of equations such as associativity and commutativity of parallel composition. Sassone and Sobociński [2003, 2005] replace them with groupoidal relative pushouts in a bicategorical setting. As noted in Remark 32, our normality assumption addresses the same difficulty by a different route and the relationship deserves to be settled rather than left implicit; our matching of labels up to $\approx$ is already partway there.

Higher-order syntax. Presenting binding algebraically goes back to Fiore et al. [1999]; Arkor and McDermott [2021] give the higher-order algebraic theories closest in spirit to our lambda theories. What we add is the treatment of rewriting as a proposition in the subobject fibration, so that types, terms, equations and rewrites are one structure and entailments may be curried to reason under binders. Di Gianantonio et al. [2009] apply RPOs to the λ-calculus using second-order contexts and supply the IPO-uniformity condition on which Theorem 26 is modelled.

Encodings. Gorla [2010] surveys the criteria by which an encoding between process calculi is judged — compositionality, name invariance, operational correspondence, divergence reflection, success sensitiveness. Definition 38 is more parsimonious and, we think, better factored: the criteria are consequences of being a functor into $[\![\mathcal{D}]\!]/\approx$ rather than a list. Milner [1992] is the origin of the question for λ into π; Boudol [1997] gives the name-passing variant that makes the translation tractable; Lybech [2022] gives the π-into-ρ translation we take up in §7. Scott-style encodings of data are folklore; Mogensen [1992] is the usual reference.

Ambients. Bonchi et al. [2009] treat the ambient calculus in the reactive-systems framework and find that the IPO semantics must be coarsened by barbs. We take this as evidence that a locality operator needs a different construction, and omit ambients accordingly (Remark 6).

10 Conclusion and open problems

We have made the class of admissible observers a parameter of the Leifer–Milner construction, shown that the parameter is determined by a choice of equivalence at the ground types, and given a congruence theorem whose statement is $\mathcal{D}$-relative on both sides. Three phenomena that look unrelated — reflection in the ρ-calculus, the restriction of an encoding's observers to encoded contexts, and the correctness of compiling away built-in data — are the same theorem with three choices of $\mathcal{D}$.

The outstanding work, in the order we would attack it:

1. **Decomposition-closure** (Obligation 28). Whether $[\![C_\pi]\!]$ is closed under decomposition up to $\approx$ inside ρ. Everything else depends on this and it is cheap to test: one worked factorization of an encoded context will settle it.

2. **The ν lemma** (Obligation 42). The single largest piece of new mathematics, and the one that has been outstanding longest. We recommend proving it against $\rho^!$ and composing with Theorem 40.

3. **Weak congruence** (Theorem 26). Every result in §7 and §8 is about weak bisimilarity, so the general statement is load-bearing; at present only the direct argument for ρ is available.

4. **Existence of relative IPOs** (Proposition 27). The codirectedness argument needs a well-foundedness condition we have not identified.

5. **Strict versus groupoidal pushouts** (Remark 32). Either justify normality as a checkable hypothesis on presentations, or redo the construction with GRPOs.

6. **The retraction** (Obligation 55), and with it `Int`, `String` and lists, which we expect to follow the pattern of §8.4 without new ideas.

7. **Cost.** Remark 57 identifies a quantity — the ledger gap of an encoding — that measures what a built-in operation buys. We have not studied it.

References

J. Adámek, J. Rosický, and E. M. Vitale. *Algebraic Theories: A Categorical Introduction to General Algebra*. Cambridge Tracts in Mathematics. Cambridge University Press, 2010. URL https://perso.uclouvain.be/enrico.vitale/gab_CUP2.pdf.

Nathanael Arkor and Dylan McDermott. Higher-order algebraic theories. 2021. URL https://arkor.co/files/Higher-order%20algebraic%20theories.pdf.

M. Barr and C. Wells. *Toposes, Triples and Theories*. Grundlehren der mathematischen Wissenschaften. Springer New York, 2013. ISBN 9781489900210. URL https://cjhb.site/Files.php/Books/%28Uncategorized%29/Before%202022/Toposes,%20Triples%20and%20Theories%20-%20M.%20Barr,%20W.%20Wells.pdf.

Filippo Bonchi, Fabio Gadducci, and Giacoma Valentina Monreale. Reactive systems, barbed semantics, and the mobile ambients. volume 5504 of *Lecture Notes in Computer Science*, pages 272–287. Springer, 2009. doi: 10.1007/978-3-642-00596-1_20. URL https://link.springer.com/content/pdf/10.1007/978-3-642-00596-1_20.pdf.

Gérard Boudol. The pi-calculus in direct style. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '97, page 228–242, New York, NY, USA, 1997. Association for Computing Machinery. ISBN 0897918533. URL https://doi.org/10.1145/263699.263726.

Pietro Di Gianantonio, Furio Honsell, and Marina Lenisa. Rpo, second-order contexts, and lambda-calculus. *Logical Methods in Computer Science*, Volume 5, Issue 3, August 2009. ISSN 1860-5974. URL https://arxiv.org/abs/0906.2727.

Marcelo Fiore, Gordon Plotkin, and Daniele Turi. Abstract syntax and variable binding. In *Logic in Computer Science (LICS)*, pages 193–202, 1999.

Daniele Gorla. Towards a unified approach to encodability and separation results for process calculi. *Information and Computation*, 208(9):1031–1053, 2010.

B. Jacobs. *Categorical Logic and Type Theory*. Number 141 in Studies in Logic and the Foundations of Mathematics. North Holland, Amsterdam, 1999. URL https://annas-archive.org/search?q=jacobs+categorical+logic.

Peter T Johnstone. *Sketches of an elephant*. Oxford logic guides. Oxford Univ. Press, New York, NY, 2002. URL https://cds.cern.ch/record/592033.

Bartek Klin, Vladimiro Sassone, and Paweł Sobociński. Labels from reductions: Towards a general theory. In *Algebra and Coalgebra in Computer Science*, pages 30–50. Springer Berlin Heidelberg, 2005. ISBN 978-3-540-31876-7. URL https://www.cs.ox.ac.uk/people/bartek.klin/papers/calco05B.pdf.

Anders Kock. Strong functors and monoidal monads. *Archiv der Mathematik*, 23:113–120, 12 1972. doi: 10.1007/BF01304852.

J. Lambek and P.J. Scott. *Introduction to Higher-Order Categorical Logic*. Cambridge Studies in Advanced Mathematics. Cambridge University Press, 1988. ISBN 9780521356534. URL https://books.google.com/books?id=6PY_emBeGjUC.

James J. Leifer and Robin Milner. Deriving bisimulation congruences for reactive systems. In Catuscia Palamidessi, editor, *CONCUR 2000 — Concurrency Theory*, pages 243–258, Berlin, Heidelberg, 2000. Springer Berlin Heidelberg. ISBN 978-3-540-44618-7. URL https://www.pure.ed.ac.uk/ws/portalfiles/portal/17894421/Leifer_and_Milner_2000_Deriving_Bisimulation_Congruences.pdf.

Stian Lybech. Encodability and separation for a reflective higher-order calculus. *Electronic Proceedings in Theoretical Computer Science*, 368:95–112, September 2022. ISSN 2075-2180. doi: 10.4204/eptcs.368.6. URL http://dx.doi.org/10.4204/EPTCS.368.6.

L.G. Meredith and Matthias Radestock. A reflective higher-order calculus. *Electronic Notes in Theoretical Computer Science*, 141(5):49–67, 2005. ISSN 1571-0661. doi: https://doi.org/10.1016/j.entcs.2005.05.016. URL https://www.sciencedirect.com/science/article/pii/S1571066105051893. Proceedings of the Workshop on the Foundations of Interactive Computation (FInCo 2005).

R. Milner. *A Calculus of Communicating Systems*. Springer-Verlag, Berlin, Heidelberg, 1982. ISBN 0387102353. URL https://www.lfcs.inf.ed.ac.uk/reports/86/ECS-LFCS-86-7/ECS-LFCS-86-7.pdf.

Robin Milner. Functions as processes. *Mathematical Structures in Computer Science*, 2(2):119–141, 1992.

Robin Milner. The polyadic pi-calculus: a tutorial. 1993. URL https://api.semanticscholar.org/CorpusID:11201988.

Robin Milner and Ole Høgh Jensen. Bigraphs and mobile processes (revised): Technical report 580. Workingpaper, February 2004. URL https://www.cl.cam.ac.uk/archive/rm135/Jensen-monograph.pdf.

Torben Æ. Mogensen. Efficient self-interpretation in lambda calculus. *Journal of Functional Programming*, 2(3):345–363, 1992.

Gordon D. Plotkin. A structural approach to operational semantics. In *J. Log. Algebr. Program.*, volume 60-61, pages 17–139, 2004.

Vladimiro Sassone and Paweł Sobociński. Deriving bisimulation congruences using 2-categories. *Nordic Journal of Computing*, 10(2):163–183, 2003.

Vladimiro Sassone and Paweł Sobociński. Locating reaction with 2-categories. *Theoretical Computer Science*, 333(1-2):297–327, 2005.

A Inference rules for lambda theories

There is a characterization theorem for subobject fibrations which licenses using their internal language as the syntax for categories with finite limits.

Theorem 59 (Jacobs, 1999, 4.9.4)**.** *Let $\pi : \mathrm{P} \to \mathrm{T}$ be a fibered preorder with finite products and equality. Then π is equivalent to a subobject fibration $\mathrm{SubT} \to \mathrm{T}$ if and only if it has extensionality*

$$\Gamma \mid (f =_B g) \vdash f = g : B$$

full subset types

$$\frac{\Gamma, x : \sigma \mid \Theta, \varphi \vdash \psi}{\Gamma, y : \{x : \sigma \mid \varphi\} \mid \Theta[\mathrm{o}(y)/x] \vdash \psi[\mathrm{o}(y)/x]}$$

and unique choice.

$$\frac{i : I, j_1, j_2 : J \mid R(i, j_1), R(i, j_2) \vdash j_1 = j_2}{i : I, j : J \vdash (\exists j.R \text{ prop}), (\exists j.R \sim R)}$$

The structural rules are summarized by the fact that each level forms a category.

$$\frac{}{x : A \vdash x : A} \qquad \frac{\Gamma \vdash a : A \quad x : A \vdash b : B}{\Gamma \vdash b(a) : B} \qquad \frac{\Gamma \vdash a_1 = a_2 : A \quad x : A \vdash b : B}{\Gamma \vdash b(a_1) = b(a_2) : B}$$

$$\frac{}{\Gamma \mid \varphi \vdash \varphi} \qquad \frac{\Gamma \mid \varphi \vdash \theta \quad \Gamma \mid \theta \vdash \psi}{\Gamma \mid \varphi \vdash \psi} \qquad \frac{\Gamma \vdash a : A \quad \Gamma, x : A \mid \theta \vdash \varphi}{\Gamma \mid \theta[a/x] \vdash \varphi[a/x]}$$

units

$$\frac{}{1 \text{ type}} \qquad \frac{}{\Gamma \vdash * : 1} \qquad \frac{\Gamma \vdash x : 1}{\Gamma \vdash x = * : 1}$$

$$\frac{}{\Gamma \vdash \top \text{ prop}} \qquad \frac{}{\Gamma \mid \varphi \vdash \top}$$

products

$$\frac{A \text{ type} \quad B \text{ type}}{A \times B \text{ type}} \qquad \frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B}{\Gamma \vdash \langle a, b \rangle : A \times B} \qquad \frac{\Gamma \vdash p : A \times B}{\Gamma \vdash \pi_1(p) : A} \qquad \frac{\Gamma \vdash p : A \times B}{\Gamma \vdash \pi_2(p) : B}$$

$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B}{\Gamma \vdash \pi_1\langle a, b \rangle = a : A} \qquad \frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B}{\Gamma \vdash \pi_2\langle a, b \rangle = b : B}$$

$$\frac{\Gamma \vdash \varphi \text{ prop} \quad \Gamma \vdash \psi \text{ prop}}{\Gamma \vdash \varphi \wedge \psi \text{ prop}} \qquad \frac{\Gamma \mid \theta \vdash \varphi \quad \Gamma \mid \theta \vdash \psi}{\Gamma \mid \theta \vdash \varphi \wedge \psi} \qquad \frac{\Gamma \mid \theta \vdash \varphi \wedge \psi}{\Gamma \mid \theta \vdash \varphi} \qquad \frac{\Gamma \mid \theta \vdash \varphi \wedge \psi}{\Gamma \mid \theta \vdash \psi}$$

functions

$$\frac{A \text{ type} \quad B \text{ type}}{A \to B \text{ type}} \qquad \frac{\Gamma, x : A \vdash f : B}{\Gamma \vdash \lambda x.f : A \to B} \qquad \frac{\Gamma \vdash a : A \quad \Gamma \vdash f : A \to B}{\Gamma \vdash f\, a : B}$$

$$\frac{\Gamma, x:A \vdash f:B \quad \Gamma \vdash a:A}{\Gamma \vdash (\lambda x.f)\, a = f[a/x] : B} \qquad \frac{\Gamma \vdash f : A \to B}{\Gamma \vdash \lambda x.(f\, x) = f : A \to B}$$

$$\frac{b:B \vdash \psi \text{ prop} \quad c:C \vdash \theta \text{ prop}}{\lambda b.c : [B \backslash C] \vdash \psi \Rightarrow \theta \text{ prop}} \qquad \frac{a:A, b:B \vdash f:C \quad a:A, b:B \mid \varphi, \psi \vdash \theta[f]}{a:A \mid \varphi \vdash (\psi \Rightarrow \theta)[\lambda b.f]}$$

equality

$$\frac{\Gamma \vdash a_1 : A \quad \Gamma \vdash a_2 : A}{\Gamma \vdash (a_1 =_A a_2) \text{ prop}} \qquad \frac{\Gamma, x:A \mid \theta \vdash f[x/y] =_B g[x/y]}{\Gamma, x:A, y:A \mid \theta, (x =_A y) \vdash f =_B g} \qquad \frac{}{\Gamma \mid (f =_B g) \vdash f = g : B}$$

subsets

$$\frac{x:A \vdash \varphi \text{ prop}}{\{x:A \mid \varphi\} \text{ type}} \qquad \frac{\Gamma \vdash a:A \quad \Gamma \mid \top \vdash \varphi[a/x]}{\Gamma \vdash i(a) : \{x:A \mid \varphi\}} \qquad \frac{\Gamma \vdash a : \{x:A \mid \varphi\}}{\Gamma \vdash o(a) : A}$$

$$\frac{\Gamma, x:A \mid \theta, \varphi \vdash \psi}{\Gamma, y:\{x:A \mid \varphi\} \mid \theta[o(y)/x] \vdash \psi[o(y)/x]}$$

B Meta-theories

Definition 60. *The **metatheory of finite limits** $\mathbb{T}_\wedge$ is a finite limit category presented as follows.*

$$\begin{aligned} &\vdash && \mathrm{T}_0 \text{ type} \\ &\vdash && \mathrm{T}_1 \text{ type} \\ f:\mathrm{T}_1 &\vdash && \mathrm{s}(f) : \mathrm{T}_0 \\ f:\mathrm{T}_1 &\vdash && \mathrm{t}(f) : \mathrm{T}_0 \end{aligned}$$

We use $f : \mathrm{T}(A, B)$ as shorthand for $A, B : \mathrm{T}_0, f : T_1, \mathrm{s}(f) = A, \mathrm{t}(f) = B$.

$$\begin{aligned} A:\mathrm{T}_0 &\vdash && \mathrm{id}_A : \mathrm{T}(A,A) \\ f:\mathrm{T}(A,B), g:\mathrm{T}(B,C) &\vdash && g \circ f : \mathrm{T}(\mathrm{s}(f), \mathrm{t}(g)) \\ f:\mathrm{T}(A,B) &\vdash && \mathrm{id}_A \circ f = f, \\ & && f \circ \mathrm{id}_B = f \\ f:\mathrm{T}(A,B), g:\mathrm{T}(B,C), h:\mathrm{T}(C,D) &\vdash && (h \circ g) \circ f = h \circ (g \circ f) \end{aligned}$$

This defines a category T; we now give it a terminal object and pullbacks.

$$\begin{aligned} &\vdash && 1 : \mathrm{T}_0 \\ A:\mathrm{T}_0 &\vdash && A_! : \mathrm{T}(A, 1) \\ f:\mathrm{T}(A,1) &\vdash && f = A_! \end{aligned}$$

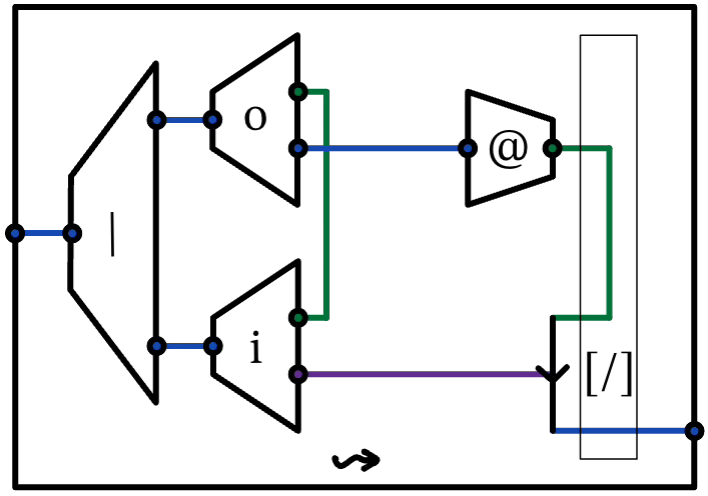
$$
\begin{array}{rcl}
p : \mathrm{T}(A, C), q : \mathrm{T}(B, C) & \vdash & p \times_C q : \mathrm{T}_0, \\
& & \pi_1 : \mathrm{T}(p \times_C q, A), \\
& & \pi_2 : \mathrm{T}(p \times_C q, B), \\
& & p \circ \pi_1 = q \circ \pi_2 \\
x : \mathrm{T}(Z, A), y : \mathrm{T}(Z, B) \,|\, p \circ x = q \circ y & \vdash & \langle x, y \rangle : \mathrm{T}(Z, p \times_C q), \\
& & \pi_1 \circ \langle x, y \rangle = x, \\
& & \pi_2 \circ \langle x, y \rangle = y \\
..., h : \mathrm{T}(Z, p \times_C q) \,|\, \pi_1 \circ h = x, \pi_2 \circ h = y & \vdash & h = \langle x, y \rangle
\end{array}
$$

Definition 61. *The meta-theory of cartesian closed categories $\mathbb{T}_\wedge^\to$ is $\mathbb{T}_\wedge$ plus the following.*

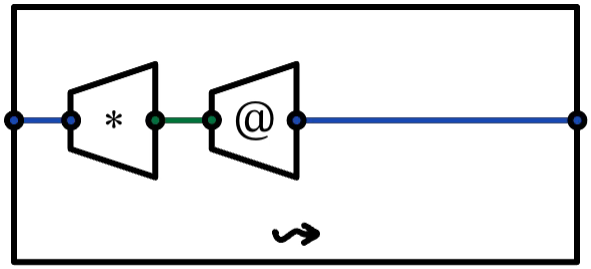
$$
\begin{array}{rcl}
A, B : \mathrm{T}_0 & \vdash & [A \to B] : \mathrm{T}_0, \\
& & \varepsilon_B^A : \mathrm{T}([A \to B] \times A, B) \\
f : \mathrm{T}(\Gamma \times A, B) & \vdash & \lambda a.f : \mathrm{T}(\Gamma, [A \to B]) \\
..., t : A & \vdash & \varepsilon_B^A(\lambda a.f, t) = f(t)
\end{array}
$$

C String diagrams

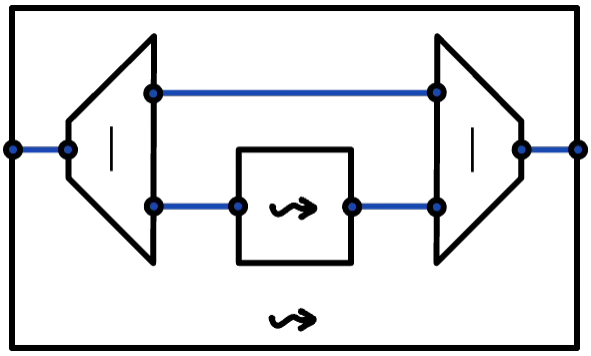
ρ calculus



$out_k(n, \vec{q})|in_k(n, \lambda\vec{x}.p) \rightsquigarrow p[@\vec{q}/\vec{x}]$

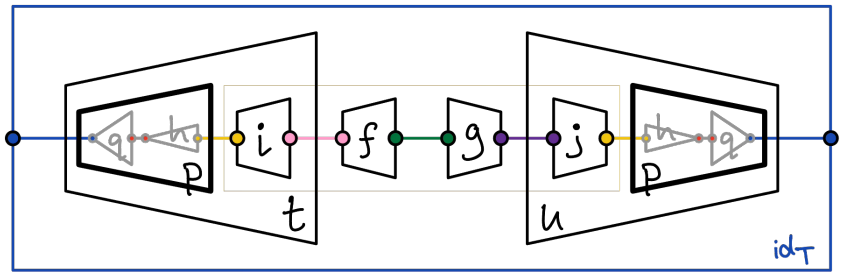


$*@p \rightsquigarrow p$

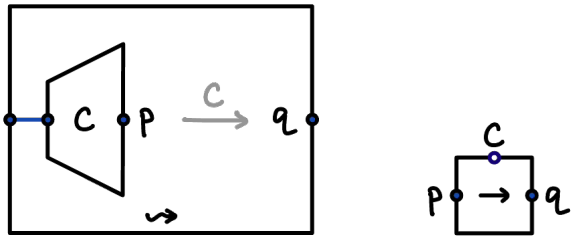


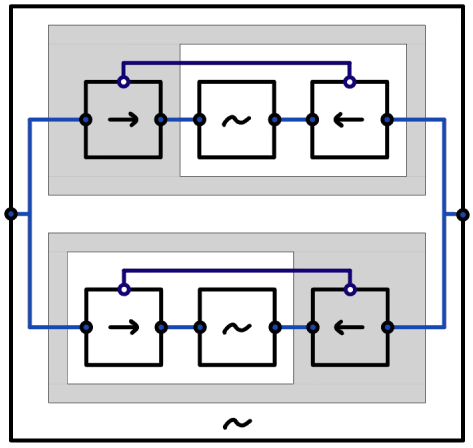
$l \rightsquigarrow r \;\Rightarrow\; p|l \rightsquigarrow p|r$

relative pushouts, contexts, bisimulation



The relative pushout of $t[f] = u[g]$ is $(p, (i, j))$, which factors uniquely through every q.





$$p_0 \sim q_0$$
$$\forall C, p_1.\ (p_0 \overset{C}{\mapsto} p_1) \Rightarrow \exists q_1.\ q_0 \overset{C}{\mapsto} q_1 \wedge p_1 \sim q_1$$
$$\forall C, q_1.\ (q_0 \overset{C}{\mapsto} q_1) \Rightarrow \exists p_1.\ p_0 \overset{C}{\mapsto} p_1 \wedge p_1 \sim q_1$$

To read each "grey bottle-opener" shape: $\forall x.P(x) \Rightarrow Q(x)$ is equivalent to $\neg[\exists x.P(x) \wedge \neg[Q(x)]]$ — the outer negation is grey, and the inner negation flips back to white.